```
// ===== src/main.js =====
// 入口：主循环、场景切换、各种弹窗（结果、包裹、员工、汇报、升级、抽造型、图鉴、成就、设置、新手引导）。
import { ctx, W, H, SAFE_TOP, onPointer, raf, audio, loadArtPackage, whenArtReady, setupShare, setupUpdate, now, vibrate
} from "./platform.js";
import { C, beginFrame, tap, rrect, text, textFit, para, paraHeight, button, image, dim, floatText, drawFloats,
  Scroller, vgrad, anim, kick, ease, wrapLines } from "./ui.js";
import { G, load, save, newPlayer, startDay, resultNext, resultAlt, openGift, upcomingDue, meta, staffData, gifts,
  upLevel, buyUpgrade, nextHireCost, hire, placesOpen, tripMinutes, sendStaff, answerDilemma, finishReport, draw,
  setLook, tickStaff, bump, staffLevel, dailyGift } from "./state.js";
import * as S from "./state.js";
import { drawHome } from "./scenes/home.js";
import { drawShop } from "./scenes/shop.js";
import { showRewardedAd, adsEnabled } from "./ads.js";
// ---------------- 场景和弹窗 ----------------
let scene = "splash";          // splash → home ⇄ shop
let overlay = null;            // 当前弹窗名
let overlayData = null;        // 弹窗用的数据
const scroller = new Scroller();
const splashAt = now();
const nav = {
  openShop() {
    if (G.runState !== "playing" || !G.deal) startDay();
    scene = "shop";
    kick("arrive", 500);
  },
  goHome() { scene = "home"; audio.bgm("shop"); },
  open(name, data = null) { overlay = name; overlayData = data; scroller.y = 0; },
  close() { overlay = null; overlayData = null; },
  openGift() {
    const r = openGift();
    if (r) { overlay = "gift"; overlayData = { ...r, t0: now() }; vibrate(); }
  },
  tapStaff(s) {
    if (s.status === "back") nav.open("report", { staff: s, page: 0, answer: null });
    else nav.open("staffCard", { staff: s });
  },
  dueText() {
    const d = upcomingDue();
    return d ? `第 ${d.day} 天要还贷 ¥${d.amount}` : "";
  },
};
// ---------------- 启动 ----------------
function boot() {
  const had = load();
  if (!had) { newPlayer(); save(); }
  if (G.runState !== "playing" && G.runState !== "day_end") G.runState = "day_end";
  setupShare();
  setupUpdate();
  loadArtPackage();
  if (!had) overlayData = { intro: true };
}
```

```
// ---------------- 输入 ----------------
let downAt = null;
onPointer("down", (x, y) => { downAt = { x, y, t: now() }; scroller.down(x, y); audio.resume(); });
onPointer("move", (x, y) => scroller.move(x, y));
onPointer("up", (x, y) => {
  const moved = scroller.up();
  if (!downAt) return;
  const far = Math.hypot(x - downAt.x, y - downAt.y) > 14;
  downAt = null;
  if (moved || far) return;                 // 滑动不算点击
  if (scene === "splash") { if (now() - splashAt > 1200) enterGame(); return; }
  tap(x, y);
});
function enterGame() {
  scene = "home";
  const daily = !(overlayData && overlayData.intro) && dailyGift();   // 第一次玩的人先看引导，见面礼已经有了
  if (daily) floatText(`${daily.staff ? daily.staff.name + ": " : ""}${daily.text}`, C.neon);
  whenArtReady(() => audio.bgm("shop"));   // 点了"开始"才放音乐（平台规范：不能自动播放）；音乐在分包里
  if (overlayData && overlayData.intro) nav.open("intro", { page: 0 });
}
// ---------------- 主循环 ----------------
function frame() {
  beginFrame();
  ctx.clearRect(0, 0, W, H);
  if (scene === "splash") drawSplash();
  else {
    // 员工到点回来：不管在哪个画面都要检查
    if (scene === "shop") for (const s of tickStaff()) floatText(`${s.name} 带着汇报回来了！`, C.neon);
    if (scene === "home") drawHome(nav); else drawShop(nav);
    if (G.result) drawResult();
    if (overlay) drawOverlay();
  }
  drawFloats();
  drawAchToasts();
  raf(frame);
}
// ---------------- 启动页：健康游戏忠告 ----------------
function drawSplash() {
  vgrad(0, 0, W, H, C.bgTop, C.bgBottom);
  const k = Math.min(1, (now() - splashAt) / 600);
  ctx.globalAlpha = k;
  image("gift_big", W / 2 - 110, H * 0.2, 220, 220);
  text("待机回收站", W / 2, H * 0.2 + 280, 72, C.pink, "center", true);
  text("什么都收的回收站 · 员工出门探险", W / 2, H * 0.2 + 350, 26, C.text, "center");
  // 文字必须和平台规定的完全一致
  const tips = ["《健康游戏忠告》", "抵制不良游戏，拒绝盗版游戏。", "注意自我保护，谨防受骗上当。",
    "适度游戏益脑，沉迷游戏伤身。", "合理安排时间，享受健康生活。"];
  tips.forEach((l, i) => text(l, W / 2, H - 300 + i * 38, i ? 23 : 25, C.muted, "center", i === 0));
  if (now() - splashAt > 1200) {
    ctx.globalAlpha = 0.6 + 0.4 * Math.sin(now() / 400);
```

```
    text("点击屏幕开始", W / 2, H * 0.2 + 440, 32, C.neon, "center", true);
  }
  ctx.globalAlpha = 1;
}
// ---------------- 结果卡片（成交 / 打烊 / 破产） ----------------
function drawResult() {
  const r = G.result;
  dim(0.6);
  const k = ease.back(Math.min(1, (now() - r.t0) / 300));
  const w = W - 80;
  const bodyH = paraHeight(r.body, w - 60, 24);
  const h = 230 + bodyH + (r.btn2 ? 76 : 0) + (adOffer(r) ? 76 : 0);
  const x = 40, y = Math.max(SAFE_TOP + 40, (H - h) / 2);
  ctx.save();
  ctx.translate(W / 2, y + h / 2); ctx.scale(0.85 + 0.15 * k, 0.85 + 0.15 * k); ctx.translate(-W / 2, -(y + h / 2));
  rrect(x, y, w, h, 24, C.paper, C.paperDark, 3);
  text(r.title, W / 2, y + 44, 30, C.ink, "center", true);
  text(r.big, W / 2, y + 104, 56, r.bigColor, "center", true);
  para(r.body, x + 30, y + 146, w - 60, 24, C.inkSoft, "center");
  let by = y + 160 + bodyH;
  button(r.btn, x + 30, by, w - 60, 62, () => resultNext(), { size: 28 });
  by += 76;
  if (r.btn2) { button(r.btn2, x + 30, by, w - 60, 56, () => resultAlt(), { fill: "rgba(0,0,0,0)", stroke: C.inkSoft, co
lor: C.inkSoft, size: 24 }); by += 76; }
  const ad = adOffer(r);
  if (ad) button(ad.label, x + 30, by, w - 60, 56, ad.fn, { fill: C.ok, size: 24 });
  ctx.restore();
}
/** 结果卡片上可选的“看广告”按钮（只在开了广告时出现）。 */
function adOffer(r) {
  if (!adsEnabled()) return null;
  if (G.runState === "lost" && !r.adUsed) {
    return { label: "看个广告，收债人再宽限一天", fn: () => showRewardedAd("revive", () => {
      r.adUsed = true;
      G.runState = "day_end";
      G.cash += 300;
      G.result = null;
      floatText("收债人：就再给你一天。", C.ok);
      save();
    }) };
  }
  return null;
}
// ---------------- 弹窗 ----------------
function drawOverlay() {
  switch (overlay) {
    case "gift": return drawGift();
    case "staff": return drawStaffPanel();
    case "staffCard": return drawStaffCard();
    case "send": return drawSendPanel();
    case "report": return drawReport();
```

```
    case "upgrades": return drawUpgrades();
    case "gacha": return drawGacha();
    case "gachaResult": return drawGachaResult();
    case "skinPick": return drawSkinPick();
    case "book": return drawBook();
    case "ach": return drawAch();
    case "settings": return drawSettings();
    case "intro": return drawIntro();
  }
}
/** 面板左上角“返回”：图鉴里先回到系列列表，其余直接关掉。 */
function panelBack() {
  if (overlay === "book" && overlayData && overlayData.series) { overlayData = null; scroller.y = 0; return; }
  if (overlay === "send" || overlay === "skinPick") { nav.open("staff"); return; }
  nav.close();
}
/** 通用面板：纸色大卡片 + 返回 + 标题。返回内容区 {x,y,w,h}。 */
function panel(title, sub = "") {
  dim(0.7);
  const x = 20, y = SAFE_TOP + 16, w = W - 40, h = H - y - 30;
  rrect(x, y, w, h, 24, C.paper, C.paperDark, 3);
  // 返回放左上角（平台规范：除结果页外左上角要有返回；右上角留给平台的胶囊菜单）
  button("‹ 返回", x + 20, y + 20, 120, 52, () => panelBack(), { fill: C.paperDark, color: C.ink, size: 24 });
  textFit(title, x + 160, y + 46, w - 180, 32, C.ink, "left", true);
  let top = y + 90;
  if (sub) { const sh = para(sub, x + 30, top, w - 60, 21, C.inkSoft); top += sh + 10; }
  return { x: x + 20, y: top, w: w - 40, h: y + h - top - 20 };
}
// ---- 包裹 ----
function drawGift() {
  const g = overlayData;
  dim(0.75);
  const w = W - 100, x = 50;
  const bodyH = paraHeight(g.body, w - 60, 25);
  const h = 430 + bodyH;
  const y = Math.max(SAFE_TOP + 40, (H - h) / 2);
  rrect(x, y, w, h, 24, C.paper, C.paperDark, 3);
  text(g.title, W / 2, y + 44, 32, g.good ? C.ink : C.stamp, "center", true);
  const k = ease.back(Math.min(1, (now() - g.t0) / 400));
  const s = 220 * (0.6 + 0.4 * k);
  image(g.art, W / 2 - s / 2, y + 80 + (220 - s) / 2, s, s);
  para(g.body, x + 30, y + 320, w - 60, 25, C.ink, "center");
  const more = G.giftQueue.length;
  button(more ? `收下，再拆一个（还有 ${more} 个）` : "收下", x + 30, y + h - 82, w - 60, 62,
    () => { if (more) nav.openGift(); else nav.close(); }, { size: 26 });
}
// ---- 员工列表 ----
function drawStaffPanel() {
  const cost = nextHireCost();
  const r = panel("员工", "派员工出门探险：到点会自己回来，带着冒险故事和包裹。可以关掉游戏，回来再收。");
```

```
  const rowH = 132;
  let y = scroller.begin(r.x, r.y, r.w, r.h);
  for (const s of G.staff) {
    rrect(r.x, y, r.w, rowH - 12, 16, "#f8efd9", "#c9b48d", 2);
    image(s.look, r.x + 10, y + 6, 90, 108, { alignY: 1 });
    const nw = text(s.name, r.x + 116, y + 34, 28, C.ink, "left", true);
    const lv = staffLevel(s);
    rrect(r.x + 126 + nw, y + 18, 24 * lv.title.length + 20, 32, 10, C.ink);
    text(lv.title, r.x + 136 + nw, y + 34, 20, C.neon, "left", true);
    let status = "在店里歇着", color = C.inkSoft;
    if (s.status === "away") {
      const place = staffData.places.find((p) => p.id === s.place);
      const left = Math.max(0, s.returnAt - now());
      status = `在${place ? place.name : "外面"}，${fmtLeft(left)}后回来`; color = "#2f6fa8";
    } else if (s.status === "back") { status = "回来了，等你听汇报！"; color = C.stamp; }
    textFit(status, r.x + 116, y + 76, r.w - 320, 22, color);
    if (s.status === "idle") button("派出门", r.x + r.w - 190, y + 30, 170, 58, () => nav.open("send", { staff: s }), { s
ize: 24 });
    else if (s.status === "back") button("听汇报", r.x + r.w - 190, y + 30, 170, 58, () => nav.open("report", { staff: s,
page: 0, answer: null }), { size: 24 });
    else button("换造型", r.x + r.w - 190, y + 30, 170, 58, () => nav.open("skinPick", { staff: s }), { fill: C.paperDar
k, color: C.ink, size: 22 });
    y += rowH;
  }
  if (cost !== null) {
    button(cost === 0 ? "招一个新员工（免费）" : `招一个新员工 ¥${cost}`, r.x, y + 6, r.w, 64, () => hire(),
      { fill: C.ok, size: 26, disabled: G.cash < cost });
    y += 84;
  } else { text("店里已经满员了", r.x + r.w / 2, y + 30, 22, C.inkSoft, "center"); y += 60; }
  scroller.end(y - r.y + scroller.y);
}
function fmtLeft(ms) {
  const m = Math.ceil(ms / 60000);
  if (m < 60) return `${m} 分钟`;
  return `${Math.floor(m / 60)} 小时${m % 60 ? ` ${m % 60} 分` : ""}`;
}
// ---- 点店里的员工：小卡片 ----
function drawStaffCard() {
  const s = overlayData.staff;
  dim(0.6);
  const x = 40, w = W - 80, h = 520, y = (H - h) / 2;
  rrect(x, y, w, h, 24, C.paper, C.paperDark, 3);
  image(s.look, x + 20, y + 20, 220, 300, { alignY: 1 });
  text(s.name, x + 260, y + 60, 34, C.ink, "left", true);
  text(`「${staffLevel(s).title}」  探险 ${s.trips || 0} 次`, x + 260, y + 300, 22, C.inkSoft);
  const lines = ["今天想去哪儿逛逛？", "老板，有什么吩咐？", "我刚擦完柜台。", "听说码头最近有好货。", "要不要派我出去转转？"];
  para(lines[s.name.length % lines.length], x + 260, y + 100, w - 290, 25, C.inkSoft);
  button("派出门探险", x + 30, y + 340, w - 60, 62, () => nav.open("send", { staff: s }), { size: 26 });
  button("换造型", x + 30, y + 420, (w - 76) / 2, 56, () => nav.open("skinPick", { staff: s }), { fill: C.paperDark, colo
r: C.ink, size: 24 });
  button("关闭", x + 46 + (w - 76) / 2, y + 420, (w - 76) / 2, 56, () => nav.close(), { fill: C.paperDark, color: C.ink,
size: 24 });
}
// ---- 选目的地 ----
function drawSendPanel() {
```

```
  const s = overlayData.staff;
  const r = panel(`派 ${s.name} 去哪儿？`, "越远越久，带回来的包裹越好。真实时间计时，关掉游戏也会照常回来。");
  let y = scroller.begin(r.x, r.y, r.w, r.h);
  const open = placesOpen();
  for (const p of staffData.places) {
    const ok = open.includes(p);
    const h = 128;
    rrect(r.x, y, r.w, h - 12, 16, ok ? "#f8efd9" : "#e3d5bb", "#c9b48d", 2);
    text(p.name, r.x + 20, y + 32, 28, ok ? C.ink : C.inkSoft, "left", true);
    const tier = { small: "小包裹", big: "大包裹", weird: "怪包裹" }[p.tier] || "包裹";
    text(ok ? `${fmtLeft(tripMinutes(p) * 60000)} · ${tier}` : `第 ${p.unlock_day} 天解锁`, r.x + r.w - 20, y + 32, 22,
      ok ? "#2f6fa8" : C.inkSoft, "right");
    textFit(p.desc, r.x + 20, y + 76, r.w - 200, 21, C.inkSoft);
    if (ok) button("出发", r.x + r.w - 150, y + 56, 130, 48, () => {
      sendStaff(s, p);
      floatText(`${s.name} 出发去${p.name}了`, C.ok);
      nav.close();
    }, { size: 24 });
    y += h;
  }
  scroller.end(y - r.y + scroller.y);
}
// ---- 员工汇报：立绘 + 一页页的故事 + 可能的小选择 ----
function drawReport() {
  const d = overlayData, s = d.staff;
  if (!s.report) { nav.close(); return; }
  dim(0.55);
  const boxH = 420, y = H - boxH - 30;
  image(s.look, 30, y - 360, 300, 400, { alignY: 1 });
  rrect(20, y, W - 40, boxH, 22, "rgba(20,14,10,0.94)", C.neon, 2);
  rrect(40, y - 26, 260, 52, 14, C.neon);
  textFit(`${staffLevel(s).title} ${s.name} · 汇报`, 170, y, 240, 26, "#1a1208", "center", true);
  const pages = pageText(s.report, W - 100, 26, boxH - 150);
  const page = Math.min(d.page, pages.length - 1);
  para(pages[page], 50, y + 50, W - 100, 26, C.text);
  const last = page >= pages.length - 1;
  const bw = (W - 100) / 2;
  if (!last) {
    button("跳过", 50, y + boxH - 82, bw - 10, 60, () => { d.page = pages.length - 1; }, { fill: "rgba(0,0,0,0)", stroke:
C.muted, color: C.muted, size: 24 });
    button("继续", 60 + bw, y + boxH - 82, bw - 10, 60, () => { d.page++; }, { size: 26 });
    return;
  }
  if (s.dilemma && !d.answer) {
    rrect(30, y - 170, W - 60, 130, 18, C.paper, C.stamp, 3);
    para(s.dilemma.text, 50, y - 154, W - 100, 23, C.ink);
    button(`花 ¥${s.dilemma.cost}`, 50, y + boxH - 82, bw - 10, 60, () => { d.answer = answerDilemma(s, true); }, { fill:
C.ok, size: 26, disabled: G.cash < s.dilemma.cost });
    button("不用了", 60 + bw, y + boxH - 82, bw - 10, 60, () => { d.answer = answerDilemma(s, false); }, { fill: C.paperD
ark, color: C.ink, size: 26 });
    return;
  }
  if (d.answer) { rrect(30, y - 120, W - 60, 90, 18, C.paper, C.ok, 3); para(d.answer, 50, y - 104, W - 100, 23, C.ink);
}
```

```
  button("收下包裹", 50, y + boxH - 82, W - 100, 60, () => {
    finishReport(s);
    nav.close();
    nav.openGift();
  }, { size: 28 });
}
function pageText(s, w, size, h) {
  const lines = wrapLines(s, w, size);
  const per = Math.max(1, Math.floor(h / (size * 1.45)));
  const out = [];
  for (let i = 0; i < lines.length; i += per) out.push(lines.slice(i, i + per).join("\n"));
  return out.length ? out : [""];
}
// ---- 升级 ----
function drawUpgrades() {
  const r = panel("店铺升级", `现金 ¥${G.cash}  升级永久保留，但花的是现在的钱——小心还不上贷款。`);
  let y = scroller.begin(r.x, r.y, r.w, r.h);
  for (const u of meta.upgrades) {
    const lv = upLevel(u.id), max = u.levels.length;
    const h = 150;
    rrect(r.x, y, r.w, h - 12, 16, "#f8efd9", "#c9b48d", 2);
    rrect(r.x + 16, y + 20, 72, 72, 16, C.ink);
    text(u.icon, r.x + 52, y + 57, 38, C.neon, "center", true);
    text(u.name, r.x + 108, y + 36, 28, C.ink, "left", true);
    text("★".repeat(lv) + "☆".repeat(max - lv), r.x + 108 + 30 * u.name.length + 16, y + 36, 22, C.neon);
    textFit(u.desc, r.x + 108, y + 74, r.w - 330, 21, C.inkSoft);
    textFit(lv > 0 ? `现在：${u.levels[lv - 1].text}` : "还没买", r.x + 108, y + 106, r.w - 330, 20, "#2e9b5f");
    if (lv < max) {
      const c = u.levels[lv].cost;
      button(`¥${c}`, r.x + r.w - 190, y + 40, 170, 60, () => buyUpgrade(u.id), { size: 26, disabled: G.cash < c });
    } else text("满级", r.x + r.w - 105, y + 70, 26, "#2e9b5f", "center", true);
    y += h;
  }
  scroller.end(y - r.y + scroller.y);
}
// ---- 抽造型 ----
function drawGacha() {
  const g = meta.gacha;
  const r = panel("抽员工造型", `现金 ¥${G.cash}  已拥有 ${Object.keys(G.skins).length} / ${g.skins.length}  重复的退 ¥${g.
dupe_refund}`);
  const total = g.rarities.reduce((s, x) => s + x.weight, 0);
  textFit("概率：" + g.rarities.map((x) => `${x.name} ${Math.round(x.weight / total * 100)}%`).join("  "), r.x, r.y + 16,
r.w, 20, C.inkSoft);
  const bw = (r.w - 16) / 2;
  const one = () => { const res = draw(1); if (res) nav.open("gachaResult", { res, t0: now() }); };
  const ten = () => { const res = draw(10); if (res) nav.open("gachaResult", { res, t0: now() }); };
  button(`抽 1 次 ¥${g.cost}`, r.x, r.y + 44, bw, 70, one, { size: 26, disabled: G.cash < g.cost });
  button(`抽 10 次 ¥${g.ten_cost}`, r.x + bw + 16, r.y + 44, bw, 70, ten, { size: 26, disabled: G.cash < g.ten_cost });
  if (adsEnabled() && !G.stats.ad_draw_today) {
    button("看广告免费抽 1 次", r.x, r.y + 126, r.w, 56, () => showRewardedAd("draw", () => {
      G.cash += g.cost; G.stats.ad_draw_today = 1; one();
    }), { fill: C.ok, size: 24 });
```

```
  }
  skinGrid(r.x, r.y + (adsEnabled() ? 196 : 130), r.w, r.y + r.h, null);
}
function skinGrid(x, y0, w, yMax, onPick) {
  const g = meta.gacha;
  const cols = 4, gap = 12, cw = (w - gap * (cols - 1)) / cols, ch = cw * 1.35;
  let y = scroller.begin(x, y0, w, yMax - y0);
  const list = onPick ? g.skins.filter((s) => G.skins[s.look]) : g.skins;
  list.forEach((s, i) => {
    const cx = x + (i % cols) * (cw + gap), cy = y + Math.floor(i / cols) * (ch + gap);
    const rar = g.rarities.find((r) => r.id === s.rarity);
    const own = !!G.skins[s.look];
    rrect(cx, cy, cw, ch, 14, own ? "#f8efd9" : "#d7c7a9", rar.color, own ? 3 : 1);
    ctx.globalAlpha = own ? 1 : 0.25;
    image(s.look, cx + 6, cy + 6, cw - 12, ch - 50, { alignY: 1 });
    ctx.globalAlpha = 1;
    textFit(own ? s.name : "? ? ? ", cx + cw / 2, cy + ch - 34, cw - 8, 18, C.ink, "center", true);
    text(rar.name, cx + cw / 2, cy + ch - 14, 15, rar.color, "center", true);
    if (onPick && own) hitCell(cx, cy, cw, ch, () => onPick(s.look));
  });
  const rows = Math.ceil(list.length / cols);
  scroller.end(rows * (ch + gap));
}
function hitCell(x, y, w, h, fn) { button("", x, y, w, h, fn, { fill: "rgba(0,0,0,0)" }); }
function drawGachaResult() {
  const { res, t0 } = overlayData;
  dim(0.82);
  text("抽到了！ ", W / 2, SAFE_TOP + 90, 44, C.neon, "center", true);
  const fresh = res.filter((r) => r.fresh).length, refund = res.reduce((s, r) => s + r.refund, 0);
  text(`新造型 ${fresh} 个${refund ? `  重复的退 ¥${refund}` : ""}`, W / 2, SAFE_TOP + 140, 24, C.text, "center");
  const g = meta.gacha;
  const single = res.length === 1;
  const cols = single ? 1 : 5, cw = single ? 360 : 124, ch = cw * 1.4, gap = 12;
  const gx = (W - (cols * cw + (cols - 1) * gap)) / 2;
  const gy = single ? (H - ch) / 2 - 20 : SAFE_TOP + 200;
  res.forEach((r, i) => {
    const k = Math.min(1, Math.max(0, (now() - t0 - i * 120) / 300));
    if (k <= 0) return;
    const cx = gx + (i % cols) * (cw + gap), cy = gy + Math.floor(i / cols) * (ch + gap);
    const rar = g.rarities.find((x) => x.id === r.skin.rarity);
    ctx.globalAlpha = k;
    rrect(cx, cy, cw, ch, 16, "#f8efd9", rar.color, r.skin.rarity === "ssr" ? 6 : 3);
    image(r.skin.look, cx + 8, cy + 8, cw - 16, ch - (single ? 90 : 56), { alignY: 1 });
    textFit((r.fresh ? "新·" : "") + r.skin.name, cx + cw / 2, cy + ch - (single ? 56 : 34), cw - 10, single ? 30 : 17,
r.fresh ? C.stamp : C.ink, "center", true);
    text(rar.name, cx + cw / 2, cy + ch - (single ? 22 : 14), single ? 24 : 15, rar.color, "center", true);
    ctx.globalAlpha = 1;
  });
  button("再抽", W / 2 - 250, H - 140, 230, 64, () => nav.open("gacha"), { fill: C.paperDark, color: C.ink, size: 26 });
  button("好的", W / 2 + 20, H - 140, 230, 64, () => nav.close(), { size: 26 });
}
```

```
function drawSkinPick() {
  const s = overlayData.staff;
  const r = panel(`给 ${s.name} 换造型`, "只能用已经抽到的造型。");
  skinGrid(r.x, r.y, r.w, r.y + r.h, (look) => { setLook(s, look); floatText(`${s.name} 换好了`, C.ok); nav.close(); });
}
// ---- 图鉴 ----
function drawBook() {
  const series = overlayData && overlayData.series;
  const all = gifts.collectibles;
  const have = all.filter((c) => G.collection[c.id]).length;
  if (!series) {
    const r = panel("收藏图鉴", `已收集 ${have} / ${all.length}  员工带回的包裹里能开出收藏品。`);
    const groups = new Map();
    for (const c of all) { const g = groups.get(c.series) || { list: [], art: c.art }; g.list.push(c); groups.set(c.seri
es, g); }
    const cols = 2, gap = 14, cw = (r.w - gap) / cols, ch = 150;
    let i = 0;
    const y0 = scroller.begin(r.x, r.y, r.w, r.h);
    for (const [name, g] of groups) {
      const cx = r.x + (i % cols) * (cw + gap), cy = y0 + Math.floor(i / cols) * (ch + gap);
      const got = g.list.filter((c) => G.collection[c.id]).length;
      rrect(cx, cy, cw, ch, 16, got === g.list.length ? "#fff4d6" : "#f8efd9", got === g.list.length ? C.neon : "#c9b48
d", 2);
      ctx.globalAlpha = got ? 1 : 0.35;
      image(g.art, cx + 8, cy + 8, 120, ch - 16);
      ctx.globalAlpha = 1;
      textFit(name, cx + 136, cy + 52, cw - 146, 22, C.ink, "left", true);
      text(`${got} / ${g.list.length}`, cx + 136, cy + 96, 22, got ? "#2e9b5f" : C.inkSoft);
      hitCell(cx, cy, cw, ch, () => { overlayData = { series: name }; scroller.y = 0; });
      i++;
    }
    scroller.end(Math.ceil(i / cols) * (ch + gap));
    return;
  }
  const list = all.filter((c) => c.series === series);
  const r = panel(series, `${list.filter((c) => G.collection[c.id]).length} / ${list.length}`);
  let y = scroller.begin(r.x, r.y, r.w, r.h);
  for (const c of list) {
    const own = G.collection[c.id];
    const h = own ? 40 + paraHeight(c.desc, r.w - 40, 20) + 20 : 64;
    rrect(r.x, y, r.w, h, 14, own ? "#f8efd9" : "#e3d5bb", "#c9b48d", 2);
    textFit(own ? `${c.name}  ${"★".repeat(c.rarity)}${own > 1 ? `  ×${own}` : ""}` : `? ? ?  ${"★".repeat(c.rarity)}`,
r.x + 20, y + 30, r.w - 40, 23,
      own ? C.ink : C.inkSoft, "left", true);
    if (own) para(c.desc, r.x + 20, y + 52, r.w - 40, 20, C.inkSoft);
    y += h + 10;
  }
  scroller.end(y - r.y + scroller.y);
}
// ---- 成就 ----
function drawAch() {
  const list = meta.achievements;
  const got = list.filter((a) => G.ach[a.id]).length;
```

```
  const r = panel("成就", `已解锁 ${got} / ${list.length}`);
  let y = scroller.begin(r.x, r.y, r.w, r.h);
  const sorted = [...list].sort((a, b) => (G.ach[b.id] ? 1 : 0) - (G.ach[a.id] ? 1 : 0));
  for (const a of sorted) {
    const own = !!G.ach[a.id];
    const secret = a.secret && !own;
    rrect(r.x, y, r.w, 104, 14, own ? "#fff4d6" : "#e8dcc4", own ? C.neon : "#c9b48d", 2);
    text(own ? "★" : "☆", r.x + 40, y + 52, 40, own ? C.neon : C.inkSoft, "center");
    text(secret ? "? ? ? " : a.name, r.x + 80, y + 34, 26, own ? C.ink : C.inkSoft, "left", true);
    textFit(secret ? "隐藏成就" : a.desc, r.x + 80, y + 70, r.w - 230, 20, C.inkSoft);
    if (!own && !secret) text(`${Math.min(G.stats[a.stat] || 0, a.goal)} / ${a.goal}`, r.x + r.w - 20, y + 52, 22, C.ink
Soft, "right");
    y += 114;
  }
  scroller.end(y - r.y + scroller.y);
}
// ---- 设置 ----
function drawSettings() {
  const r = panel("设置");
  let y = r.y + 20;
  const row = (label, key) => {
    text(label, r.x + 10, y + 30, 28, C.ink);
    const v = G.settings[key];
    for (let i = 0; i <= 4; i++) {
      const on = v >= i / 4 - 0.01 && i > 0;
      button(i === 0 ? "关" : "", r.x + 160 + i * 96, y, 84, 60, () => {
        G.settings[key] = i / 4; audio.setVolume(G.settings.music, G.settings.sfx); save();
      }, { fill: i === 0 ? (v === 0 ? C.stamp : C.paperDark) : on ? C.neon : C.paperDark, color: C.ink, size: 22 });
    }
    y += 90;
  };
  row("音乐", "music");
  row("音效", "sfx");
  y += 20;
  para("关于\n待机回收站：开一家什么都收的回收站。验货、砍价、揭穿编故事的顾客；派员工出门探险，带回冒险故事和包裹。\n\n" +
    "隐私说明\n本游戏不收集任何个人信息，存档只保存在你自己的手机上。", r.x + 10, y, r.w - 20, 22, C.inkSoft);
  y = r.y + r.h - 70;
  button("重新看新手引导", r.x, y, r.w, 60, () => nav.open("intro", { page: 0 }), { fill: C.paperDark, color: C.ink, size:
24 });
}
// ---- 新手引导 ----
const INTRO = [
  { art: "lounge_bg", title: "欢迎来到待机回收站", body: "这是你的店。大本营里住着你的员工。\n点下方吧台就开门营业。" },
  { art: "shop_bg", title: "收货：先验货，再砍价", body: "顾客会讲东西的来历。点“检查”看线索—\n有的人在撒谎。发现破绽就“质疑来
历”，\n揭穿了他只能按实价卖给你。" },
  { art: "gift_small", title: "派员工出门探险", body: "点上方“员工”，派他们去跳蚤市场、码头、拍卖行……\n按真实时间回来，带着冒险
故事和包裹。\n关掉游戏也照常计时。" },
  { art: "gift_big", title: "七天还清贷款", body: "第 3 天和第 7 天要还贷。\n打烊时把收来的货卖给收购商，符合“今日订单”的卖得更
贵。\n赚了钱就升级店铺、抽员工造型、集收藏品。" },
];
function drawIntro() {
  const d = overlayData;
  const p = INTRO[d.page];
  dim(0.8);
  const x = 40, w = W - 80, h = 680, y = Math.max(SAFE_TOP + 30, (H - h) / 2);
```

```
  rrect(x, y, w, h, 24, C.paper, C.paperDark, 3);
  ctx.save(); ctx.beginPath(); ctx.rect(x + 20, y + 20, w - 40, 320); ctx.clip();
  image(p.art, x + 20, y + 20, w - 40, 320, { cover: p.art.endsWith("_bg") });
  ctx.restore();
  text(p.title, W / 2, y + 390, 32, C.ink, "center", true);
  para(p.body, x + 30, y + 430, w - 60, 24, C.inkSoft, "center");
  for (let i = 0; i < INTRO.length; i++) {
    ctx.beginPath(); ctx.arc(W / 2 + (i - (INTRO.length - 1) / 2) * 28, y + h - 110, 7, 0, Math.PI * 2);
    ctx.fillStyle = i === d.page ? C.neon : C.paperDark; ctx.fill();
  }
  const last = d.page >= INTRO.length - 1;
  button(last ? "开始! " : "下一页", x + 30, y + h - 84, w - 60, 62, () => {
    if (last) { nav.close(); bump("tutorial"); if (G.giftQueue.length) floatText("员工给你留了一个见面礼，点右上角包裹", C.n
eon); }
    else d.page++;
  }, { size: 28 });
}
// ---------------- 成就弹出 ----------------
function drawAchToasts() {
  const t = now();
  G.achToasts = G.achToasts.filter((x) => t - x.t0 < 3200);
  G.achToasts.slice(0, 2).forEach((x, i) => {
    const k = Math.min(1, (t - x.t0) / 300), out = Math.max(0, (t - x.t0 - 2800) / 400);
    // 放在顶部中间（胶囊菜单下面），不挡对话框底部的按钮
    const y = SAFE_TOP + 90 + i * 100 - (1 - k) * 60;
    const bx = W / 2 - 200;
    ctx.globalAlpha = 1 - out;
    rrect(bx, y, 400, 84, 16, "rgba(20,14,10,0.95)", C.neon, 2);
    text("★", bx + 40, y + 42, 40, C.neon, "center");
    text("成就解锁", bx + 80, y + 26, 18, C.muted);
    textFit(x.a.name, bx + 80, y + 58, 300, 26, C.neon, "left", true);
    ctx.globalAlpha = 1;
  });
}
boot();
raf(frame);
// 浏览器试玩时方便调试（小游戏里没有 window，不会生效）
if (typeof window !== "undefined" && typeof wx === "undefined" && typeof tt === "undefined") {
  window.__ss = { G, S, nav, frame, get scene() { return scene; }, get overlay() { return overlay; }, enterGame };
}
// ===== src/ads.js =====
// 激励视频广告。广告位 ID 要在平台后台开通“流量主”后才拿得到，填进 data/ads.json；
// 没填的时候 adsEnabled() 为 false，游戏里不会出现任何“看广告”按钮。
import ads from "../build/data/ads.json";
import { PLATFORM } from "./platform.js";
import { floatText, C } from "./ui.js";
const api = typeof wx !== "undefined" ? wx : typeof tt !== "undefined" ? tt : null;
const unitId = () => (ads[PLATFORM] || {}).rewarded || "";
export function adsEnabled() { return !!(api && api.createRewardedVideoAd && unitId()); }
let ad = null, pending = null;
function getAd() {
```

```
  if (ad) return ad;
  ad = api.createRewardedVideoAd({ adUnitId: unitId() });
  ad.onClose((res) => {
    const done = !res || res.isEnded;   // 看完才给奖励
    const fn = pending; pending = null;
    if (done && fn) fn(); else if (fn) floatText("没看完，奖励没拿到", C.muted);
  });
  ad.onError(() => { pending = null; });
  return ad;
}
/** 播放激励视频；看完后调用 onReward。placement 只用来区分用途。 */
export function showRewardedAd(placement, onReward) {
  if (!adsEnabled()) return;
  const a = getAd();
  pending = onReward;
  a.show().catch(() => a.load().then(() => a.show())).catch(() => { pending = null; floatText("广告暂时没准备好", C.muted);
}));
}
// ===== src/deal.js =====
// 一笔交易的全部规则（从电脑版 src/deal.gd 原样移植）：顾客性格、货物真实价值、鉴定、质疑来历、砍价。
// 数值来自 data/tuning.json，台词来自 data/personalities.json。
export const rand = (a, b) => a + Math.random() * (b - a);
export const randi = (a, b) => Math.floor(rand(a, b + 1));
export const pick = (arr) => arr[Math.floor(Math.random() * arr.length)];
const round10 = (v) => Math.max(10, Math.round(v / 10) * 10);
function pickWeighted(rows) {
  const total = rows.reduce((s, r) => s + Number(r[2]), 0);
  let roll = Math.random() * total;
  for (const r of rows) { roll -= Number(r[2]); if (roll <= 0) return r; }
  return rows[0];
}
const meanMult = (variants) => Math.max(0.1, variants.reduce((s, v) => s + Number(v[1]), 0) / Math.max(1, variants.lengt
h));
export class Deal {
  constructor(tuning, catalog, archetypes, personas, reputation) {
    this.tuning = tuning;
    const arch = pick(archetypes);
    this.customer = { name: pick(arch.names), type: arch.type, art: arch.art };
    this.persona = personas[this.customer.type];
    this.item = this.makeItem(catalog);
    this.challengeCount = 0;
    this.exposed = false;
    const lieChance = tuning.lie_chance * this.persona.lie_mult - reputation * tuning.rep_lie_reduction;
    this.lying = Math.random() < Math.min(0.9, Math.max(0, lieChance));
    const cat = catalog.categories[this.item.tag];
    const lie = this.lying ? pick(cat.lies) : null;
    if (this.lying) this.story = lie.story;
    else if (Math.random() < 0.45) this.story = pick(catalog.meme_stories);
    else this.story = pick(cat.stories);
    // 抽 3 个检查部位；撒谎时一定包含谎言对应的部位
    let parts = Object.keys(cat.parts).sort(() => Math.random() - 0.5).slice(0, 3);
    if (this.lying && !parts.includes(lie.part)) parts[randi(0, 2)] = lie.part;
```

```
    let value = this.item.base;
    this.clues = parts.map((part) => {
      if (this.lying && part === lie.part) return { part, text: lie.text, mult: 1, revealed: false, isLie: true };
      const v = pick(cat.parts[part]);
      if (v.length > 2 && v[2] === "leaky") this.item.leaky = true;
      value *= Number(v[1]);
      return { part, text: v[0], mult: Number(v[1]), revealed: false, isLie: false };
    });
    // 以“这个部位的平均结果”为基准：好线索让真实价值高于参考价，坏线索低于参考价
    for (const part of parts) value /= meanMult(cat.parts[part]);
    if (this.lying) value *= Number(lie.mult);
    this.trueValue = Math.max(10, Math.round(value * rand(0.9, 1.1)));
    const greed = rand(tuning.greed_min, tuning.greed_max) + Number(this.persona.greed_add);
    this.ask = round10(this.item.base * greed);
    let floorRatio = rand(tuning.min_price_min, tuning.min_price_max) + Number(this.persona.floor_add);
    if (this.lying) floorRatio *= tuning.liar_min_factor;
    this.minPrice = Math.min(round10(this.trueValue * floorRatio), this.ask);
    this.patience = tuning.patience_start * Number(this.persona.patience_mult);
    this.roundsLeft = tuning.haggle_rounds;
  }
  makeItem(catalog) {
    const row = pick(catalog.items);
    const cond = pickWeighted(catalog.conditions);
    return {
      name: String(cond[0]) + String(row[0]), tag: row[1],
      base: Math.max(10, Math.round(Number(row[2]) * Number(cond[1]))),
      art: catalog.categories[row[1]].art, leaky: false,
    };
  }
  line(kind) {
    const pool = this.persona[kind] || [""];
    return String(pick(pool)).replace("{ask}", String(this.ask));
  }
  reveal(i) { this.clues[i].revealed = true; return `${this.clues[i].part}：${this.clues[i].text}`; }
  /** 质疑来历。mood：caught（拿出证据揭穿）/ confess（心虚招认）/ unproven（没证据）/ honest（冤枉了老实人） */
  challenge() {
    const t = this.tuning;
    this.challengeCount++;
    if (this.lying && !this.exposed) {
      const lieSeen = this.clues.some((c) => c.isLie && c.revealed);
      const confess = this.challengeCount >= 2 && Math.random() < t.confess_chance;
      if (lieSeen || confess) {
        this.exposed = true;
        // 被揭穿就说实话：要价降到真实价值，底价再低一截，揭穿骗子一定有赚头
        this.minPrice = round10(this.trueValue * t.exposed_floor_ratio);
        this.ask = Math.max(this.minPrice, round10(this.trueValue * t.exposed_ask_ratio));
        return { text: `“${this.line("caught")}”`, mood: lieSeen ? "caught" : "confess" };
      }
      this.patience -= t.challenge_unproven_loss;
      return { text: `“${this.line("deny")}”`, mood: "unproven" };
```

```
    }
    this.patience -= t.challenge_honest_loss * this.challengeCount;
    return { text: `“${this.line("offended")}”`, mood: "honest" };
  }
  /** 出价。result: accept / counter / leave */
  offer(amount) {
    const t = this.tuning;
    if (amount >= this.minPrice) return { result: "accept", text: `“${this.line("accept")}”` };
    this.roundsLeft--;
    const gap = (this.minPrice - amount) / this.minPrice;
    this.patience -= t.patience_base_loss + t.patience_gap_loss * gap;
    if (this.patience <= 0 || this.roundsLeft <= 0) return { result: "leave", text: `“${this.line("leave")}”` };
    this.ask = Math.max(round10(Math.max(this.minPrice * t.counter_floor, (amount + this.ask) / 2)), this.minPrice);
    return { result: "counter", text: `“${this.line("counter")}”` };
  }
  profit(paid) { return this.trueValue - paid; }
}
// ===== src/platform.js =====
// 平台层：把微信（wx）、抖音（tt）和浏览器的差别都藏在这里。
// 游戏其余部分只用这里导出的 P，不直接碰 wx / tt / window。
const api = typeof wx !== "undefined" ? wx : typeof tt !== "undefined" ? tt : null;
export const PLATFORM = typeof wx !== "undefined" ? "wechat" : typeof tt !== "undefined" ? "douyin" : "web";
/** 逻辑分辨率：宽固定 720，高按屏幕比例。所有界面坐标都用逻辑像素。 */
export const W = 720;
function sysInfo() {
  if (api) {
    const s = (api.getWindowInfo ? api.getWindowInfo() : api.getSystemInfoSync());
    let safeTop = (s.safeArea && s.safeArea.top) || s.statusBarHeight || 0;
    // 右上角是平台的胶囊菜单（…和关闭），我们的按钮不能和它重叠：顶部从胶囊下面开始
    let menuBottom = 0;
    try {
      const m = api.getMenuButtonBoundingClientRect && api.getMenuButtonBoundingClientRect();
      if (m && m.bottom) menuBottom = m.bottom;
    } catch {}
    // 拿不到胶囊位置时（有的平台或版本没有这个接口），按常见胶囊高度留出来
    safeTop = Math.max(safeTop, menuBottom ? menuBottom + 6 : (s.statusBarHeight || 20) + 44);
    return { w: s.windowWidth, h: s.windowHeight, dpr: s.pixelRatio || 2, safeTop };
  }
  return { w: window.innerWidth, h: window.innerHeight, dpr: window.devicePixelRatio || 1, safeTop: 0 };
}
const info = sysInfo();
export const SCALE = info.w / W;            // 逻辑像素 → 屏幕像素
export const H = Math.round(info.h / SCALE);
export const SAFE_TOP = Math.round(info.safeTop / SCALE);
let canvas;
if (api) {
  canvas = api.createCanvas();               // 小游戏里第一次创建的就是屏幕上的画布
} else {
  canvas = document.createElement("canvas");
  canvas.style.width = info.w + "px";
```

```
  canvas.style.height = info.h + "px";
  document.body.appendChild(canvas);
}
canvas.width = Math.round(info.w * info.dpr);
canvas.height = Math.round(info.h * info.dpr);
export const ctx = canvas.getContext("2d");
const RATIO = SCALE * info.dpr;
export function resetTransform() { ctx.setTransform(RATIO, 0, 0, RATIO, 0, 0); }
resetTransform();
// ---------------- 触摸：统一成 down / move / up（逻辑坐标） ----------------
const handlers = { down: [], move: [], up: [] };
export function onPointer(kind, fn) { handlers[kind].push(fn); }
function emit(kind, x, y) { for (const fn of handlers[kind]) fn(x / SCALE, y / SCALE); }
if (api) {
  api.onTouchStart((e) => { const t = e.changedTouches[0]; emit("down", t.clientX, t.clientY); });
  api.onTouchMove((e) => { const t = e.changedTouches[0]; emit("move", t.clientX, t.clientY); });
  api.onTouchEnd((e) => { const t = e.changedTouches[0]; emit("up", t.clientX, t.clientY); });
  api.onTouchCancel && api.onTouchCancel((e) => { const t = e.changedTouches[0]; emit("up", t.clientX, t.clientY); });
} else {
  const pos = (e) => { const r = canvas.getBoundingClientRect(); return [e.clientX - r.left, e.clientY - r.top]; };
  let pressed = false;
  canvas.addEventListener("pointerdown", (e) => { pressed = true; emit("down", ...pos(e)); });
  window.addEventListener("pointermove", (e) => { if (pressed) emit("move", ...pos(e)); });
  window.addEventListener("pointerup", (e) => { if (pressed) { pressed = false; emit("up", ...pos(e)); } });
}
// ---------------- 存储 ----------------
export const storage = {
  get(key, fallback = null) {
    try {
      const raw = api ? api.getStorageSync(key) : localStorage.getItem(key);
      return raw ? JSON.parse(raw) : fallback;
    } catch { return fallback; }
  },
  set(key, value) {
    try {
      const raw = JSON.stringify(value);
      if (api) api.setStorageSync(key, raw); else localStorage.setItem(key, raw);
    } catch {}
  },
};
// ---------------- 图片 ----------------
// 图片和背景音乐放在分包 "res" 里（微信首包限制 4MB），启动时先加载分包；背景图是 jpg，其余 png。
const images = new Map();
let artReady = !api;
const readyFns = [];
/** 分包加载好以后执行（浏览器里立刻执行）。 */
export function whenArtReady(fn) { if (artReady) fn(); else readyFns.push(fn); }
export function loadArtPackage() {
  return new Promise((resolve) => {
    if (!api || !api.loadSubpackage) { artReady = true; resolve(); return; }
```

```
    api.loadSubpackage({
      name: "res",
      success: () => { artReady = true; readyFns.splice(0).forEach((f) => f()); resolve(); },
      fail: () => { artReady = true; readyFns.splice(0).forEach((f) => f()); resolve(); },  // 加载失败也继续，没图的地方画色块
    });
  });
}
/** 取一张图；还没加载好时返回 null（下一帧再画）。name 不带扩展名。 */
export function img(name) {
  if (!name || !artReady) return null;
  let rec = images.get(name);
  if (!rec) {
    const image = api ? api.createImage() : new Image();
    rec = { image, ok: false };
    image.onload = () => { rec.ok = true; };
    image.onerror = () => { rec.failed = true; };
    image.src = `res/art/${name}.${name.endsWith("_bg") ? "jpg" : "png"}`;
    images.set(name, rec);
  }
  return rec.ok ? rec.image : null;
}
// ---------------- 声音 ----------------
const sfxPool = new Map();
let bgmCtx = null, bgmName = "";
export const audio = {
  music: 0.6, sfxVol: 1,
  sfx(name) {
    if (this.sfxVol <= 0) return;
    try {
      if (api) {
        const a = api.createInnerAudioContext();
        a.src = `audio/sfx_${name}.mp3`;
        a.volume = this.sfxVol;
        a.onEnded(() => a.destroy());
        a.onError(() => a.destroy());
        a.play();
      } else {
        let el = sfxPool.get(name);
        if (!el) { el = new Audio(`audio/sfx_${name}.mp3`); sfxPool.set(name, el); }
        const c = el.cloneNode();
        c.volume = this.sfxVol;
        c.play().catch(() => {});
      }
    } catch {}
  },
  bgm(name) {
    if (name === bgmName && bgmCtx) return;
    bgmName = name;
    try {
      if (bgmCtx) { bgmCtx.stop ? bgmCtx.stop() : bgmCtx.pause(); if (bgmCtx.destroy) bgmCtx.destroy(); }
```

```
      bgmCtx = null;
      if (!name) return;
      if (api) {
        bgmCtx = api.createInnerAudioContext();
        bgmCtx.src = `res/audio/bgm_${name}.mp3`;
        bgmCtx.loop = true;
        bgmCtx.volume = this.music * 0.5;
        bgmCtx.play();
      } else {
        bgmCtx = new Audio(`res/audio/bgm_${name}.mp3`);
        bgmCtx.loop = true;
        bgmCtx.volume = this.music * 0.5;
        bgmCtx.play().catch(() => {});
      }
    } catch {}
  },
  setVolume(music, sfx) {
    this.music = music; this.sfxVol = sfx;
    if (bgmCtx) bgmCtx.volume = music * 0.5;
  },
  /** 浏览器要先点一下才能出声；切回前台时恢复背景音乐。 */
  resume() {
    if (bgmCtx && bgmCtx.paused !== false && bgmCtx.play) {
      const p = bgmCtx.play(); if (p && p.catch) p.catch(() => {});
    }
  },
};
if (api && api.onShow) api.onShow(() => audio.resume());
if (!api) {
  document.addEventListener("pointerdown", () => audio.resume());
  document.addEventListener("visibilitychange", () => { if (!document.hidden) audio.resume(); });
}
// ---------------- 其他 ----------------
export function vibrate() { try { api && api.vibrateShort && api.vibrateShort({ type: "light" }); } catch {} }
export const now = () => Date.now();
export function raf(fn) { return api ? requestAnimationFrame(fn) : window.requestAnimationFrame(fn); }
/** 新版本：平台在后台下载好新包后，问玩家要不要马上重启更新（存档在本地，更新后还在）。 */
export function setupUpdate() {
  if (!api || !api.getUpdateManager) return;
  try {
    const um = api.getUpdateManager();
    um.onUpdateReady(() => {
      api.showModal({
        title: "有新版本", content: "新版本已经准备好，重启一下就能用。存档不会丢。", confirmText: "重启", cancelText: "等会
儿",
        success: (r) => { if (r.confirm) um.applyUpdate(); },
      });
    });
  } catch {}
}
/** 分享（右上角菜单“转发”） */
```

```
export function setupShare() {
  if (!api) return;
  try {
    api.showShareMenu && api.showShareMenu({ withShareTicket: false, menus: ["shareAppMessage", "shareTimeline"] });
    const card = { title: "开一家什么都收的回收站，揭穿骗子、等员工带宝贝回来", imageUrl: "share.jpg" };
    api.onShareAppMessage && api.onShareAppMessage(() => card);
    api.onShareTimeline && api.onShareTimeline(() => ({ title: card.title, imageUrl: card.imageUrl }));
  } catch {}
}
// ===== src/scenes/home.js =====
// 大本营：员工在店里走来走去；出门的人不在；回来的人在吧台前排队等你听汇报。点吧台开店。
import { ctx, W, H, SAFE_TOP, img } from "../platform.js";
import { C, rrect, text, textFit, button, image, hit, vgrad } from "../ui.js";
import { G, tickStaff, staffData } from "../state.js";
import { floatText } from "../ui.js";
// 房间图里标定好的位置（和电脑版 home_room.gd 一样，按图片比例 0~1）
const DOOR = { x: 0.5, y: 0.285 };
const FLOOR = { x: 0.22, y: 0.37, w: 0.52, h: 0.24 };
const BAR_TOP = 0.70;
const DEPTH = [0.29, 0.68];
const SCALE = [0.7, 1.08];
const STAFF_H = 230;
const QUEUE = [[0.5, 0.665], [0.27, 0.665], [0.73, 0.665], [0.385, 0.575], [0.615, 0.575]];
const actors = new Map();   // 员工 id → 走路状态（不存档）
/** 房间图的位置：贴底，铺满顶栏以下的高度（太高就左右裁掉一点）。 */
export function roomRect() {
  const im = img("lounge_bg");
  const iw = im ? im.width : 1024, ih = im ? im.height : 1536;
  const h = Math.max(W * ih / iw, H - SAFE_TOP - 150);
  const w = h * iw / ih;
  return { x: (W - w) / 2, y: H - h, w, h };
}
const toScreen = (r, p) => ({ x: r.x + p.x * r.w, y: r.y + p.y * r.h });
const randomFloor = () => ({ x: FLOOR.x + Math.random() * FLOOR.w, y: FLOOR.y + Math.random() * FLOOR.h });
function actorFor(s) {
  let a = actors.get(s.id);
  if (!a) {
    a = { pos: { ...DOOR }, target: randomFloor(), wait: Math.random() * 1.5, t: Math.random() * 10, alpha: 0 };
    actors.set(s.id, a);
  }
  return a;
}
/** 员工在房间里的位置更新。dt 秒。 */
function stepActors(dt, r) {
  const present = G.staff.filter((s) => s.status !== "away");
  const queue = present.filter((s) => s.status === "back");
  for (const [id] of actors) if (!present.find((s) => s.id === id)) actors.delete(id);
  for (const s of present) {
    const a = actorFor(s);
    a.t += dt;
```

```
    a.alpha = Math.min(1, a.alpha + dt * 3);
    const qi = queue.indexOf(s);
    if (qi >= 0) a.target = qi < QUEUE.length ? { x: QUEUE[qi][0], y: QUEUE[qi][1] } : DOOR;
    if (a.wait > 0) { a.wait -= dt; continue; }
    const dx = a.target.x - a.pos.x, dy = a.target.y - a.pos.y;
    const dist = Math.hypot(dx * r.w, dy * r.h);
    if (dist > 3) {
      const step = Math.min(dist, 120 * (r.w / 720) * dt);
      a.pos.x += dx * r.w / dist * step / r.w;
      a.pos.y += dy * r.h / dist * step / r.h;
      a.walking = true;
      a.flip = dx < 0;
    } else {
      a.walking = false;
      if (qi < 0) { a.wait = 1.2 + Math.random() * 2.3; a.target = randomFloor(); }
    }
  }
  return { present, queue };
}
let last = Date.now();
export function drawHome(nav) {
  const t = Date.now();
  const dt = Math.min(0.1, (t - last) / 1000);
  last = t;
  for (const s of tickStaff()) floatText(`${s.name} 带着汇报回来了！`, C.neon);
  vgrad(0, 0, W, H, "#140d09", "#0e0b09");
  const r = roomRect();
  image("lounge_bg", r.x, r.y, r.w, r.h, { placeholder: "#3a2618" });
  const { present, queue } = stepActors(dt, r);
  // 员工：远的先画
  const order = [...present].sort((a, b) => actorFor(a).pos.y - actorFor(b).pos.y);
  for (const s of order) {
    const a = actorFor(s);
    const p = toScreen(r, a.pos);
    const depth = Math.max(0, Math.min(1, (a.pos.y - DEPTH[0]) / (DEPTH[1] - DEPTH[0])));
    const sc = (SCALE[0] + (SCALE[1] - SCALE[0]) * depth) * (r.w / 720);
    const h = STAFF_H * sc, w = h * 0.66;
    const bob = a.walking ? -Math.abs(Math.sin(a.t * 11)) * 7 : 0;
    const qi = queue.indexOf(s);
    if (qi >= QUEUE.length) continue;   // 排不下的在门外等
    ctx.globalAlpha = a.alpha;
    const im = img(s.look);
    if (im) {
      ctx.save();
      ctx.translate(p.x, p.y + bob);
      if (a.flip) ctx.scale(-1, 1);
      const s2 = Math.min(w / im.width, h / im.height);
      ctx.drawImage(im, -im.width * s2 / 2, -im.height * s2, im.width * s2, im.height * s2);
      ctx.restore();
    } else rrect(p.x - w / 4, p.y - h, w / 2, h, 20, "#7a5d44");
```

```
    // 名牌：排队时只有最前面那位写全名，其余只写“汇报”
    const back = s.status === "back";
    const label = back ? (qi === 0 ? `${s.name} · 汇报` : "汇报") : s.name;
    ctx.font = "bold 22px sans-serif";
    const tw = Math.max(60, ctx.measureText(label).width + 28);
    const ty = p.y - h - 22 + bob;
    rrect(p.x - tw / 2, ty - 18, tw, 36, 12, back ? C.neon : "rgba(255,250,240,0.92)");
    text(label, p.x, ty, 22, back ? "#1a1208" : C.ink, "center", true);
    if (back && qi === 0) {
      const k = Math.sin(t / 150) * 6;
      text("!", p.x, ty - 44 + k, 44, C.bad, "center", true);
    }
    ctx.globalAlpha = 1;
    hit(p.x - w / 2, p.y - h, w, h + 10, () => nav.tapStaff(s));
  }
  const outside = queue.length - QUEUE.length;
  if (outside > 0) {
    const d = toScreen(r, DOOR);
    text(`门外还有 ${outside} 人排队`, W / 2, d.y + 30, 24, C.text, "center", true);
  }
  // 吧台：点击开店
  const barY = r.y + BAR_TOP * r.h;
  hit(0, barY, W, H - barY, () => nav.openShop());
  const k = 0.65 + 0.35 * Math.sin(t / 450);
  ctx.globalAlpha = k;
  text("点吧台开店", W / 2, r.y + (BAR_TOP + 0.075) * r.h, 28, C.neon, "center", true);
  ctx.globalAlpha = 1;
  if (!present.length) {
    const away = G.staff.filter((s) => s.status === "away").length;
    text(away ? "员工们都出门探险了，到点就回来" : "点上方“员工”派他们出门探险", W / 2, r.y + 0.47 * r.h, 24, C.text, "cente
r", true);
  }
  drawTopBar(nav);
}
function drawTopBar(nav) {
  const top = SAFE_TOP + 10;
  vgrad(0, 0, W, top + 210, "rgba(20,13,9,1)", "rgba(20,13,9,0.85)");
  text("待机回收站", W / 2, top + 40, 46, C.pink, "center", true);
  const due = nav.dueText();
  text(`第 ${G.day} 天  ¥${G.cash}  口碑 ${G.rep}`, W / 2, top + 88, 26, C.text, "center");
  if (due) text(due, W / 2, top + 120, 22, C.muted, "center");
  const away = G.staff.filter((s) => s.status === "away").length;
  const back = G.staff.filter((s) => s.status === "back").length;
  const btns = [
    ["员工" + (back ? `·${back}` : away ? `·外出${away}` : ""), () => nav.open("staff")],
    ["升级", () => nav.open("upgrades")],
    ["抽造型", () => nav.open("gacha")],
    ["图鉴", () => nav.open("book")],
    ["成就", () => nav.open("ach")],
    ["设置", () => nav.open("settings")],
  ];
```

```
  const bw = (W - 24 - 8 * 5) / 6;
  btns.forEach(([label, fn], i) => {
    button(label, 12 + i * (bw + 8), top + 140, bw, 56, fn,
      { fill: "rgba(0,0,0,0.35)", color: i === 0 && back ? C.neon : C.text, stroke: i === 0 && back ? C.neon : "#5a4a3
e", size: 22 });
  });
  if (G.giftQueue.length) {
    button(`📦 包裹 ×${G.giftQueue.length}`, W - 220, top + 210, 200, 56, () => nav.openGift(), { size: 24 });
  }
}
export function homeQueueFront() {
  return G.staff.find((s) => s.status === "back") || null;
}
export { staffData };
// ===== src/scenes/shop.js =====
// 开店画面：顾客上门 → 看线索 → 质疑来历 → 出价砍价 → 成交或拒绝。布局照电脑版：顶栏、店内场景、当票、收银机。
import { ctx, W, H, SAFE_TOP } from "../platform.js";
import { C, rrect, text, textFit, para, paraHeight, button, image, hit, vgrad, anim, kick, shakeX, ease } from "../ui.j
s";
import { G, tuning, revealClue, challenge, quickOffer, stepOffer, makeOffer, takeAsk, refuse, upValue } from "../state.j
s";
export function drawShop(nav) {
  vgrad(0, 0, W, H, C.bgTop, C.bgBottom);
  const d = G.deal;
  let y = SAFE_TOP + 8;
  y = drawHud(nav, y);
  // 剩下的高度分给：场景（可伸缩）+ 当票 + 收银机
  const ticketH = 112 + d.clues.length * 70 + 76;
  const registerH = 318;
  const sceneH = Math.max(200, Math.min(520, H - y - ticketH - registerH - 70));
  drawScene(nav, 16, y, W - 32, sceneH);
  y += sceneH + 12;
  drawTicket(16, y, W - 32, ticketH);
  y += ticketH + 4;
  textFit(G.log.text, W / 2, y + 24, W - 40, 24, G.log.color, "center");
  drawRegister(16, H - registerH - 12, W - 32, registerH);
}
function drawHud(nav, y) {
  button("‹ 大本营", 12, y, 140, 52, () => nav.goHome(), { fill: "rgba(0,0,0,0)", stroke: "#4a3f36", color: C.text, size:
22 });
  rrect(162, y + 2, 120, 48, 24, null, C.neon, 2);
  text(`第 ${G.day} 天`, 222, y + 27, 24, C.neon, "center", true);
  text("待机回收站", 400, y + 27, 34, C.pink, "center", true);
  ctx.beginPath(); ctx.arc(560, y + 27, 13, 0, Math.PI * 2); ctx.fillStyle = "#f2c14e"; ctx.fill();
  ctx.strokeStyle = "#b8862a"; ctx.lineWidth = 3; ctx.stroke();
  textFit(`¥${G.cash}`, 582, y + 27, W - 594, 28, C.text, "left", true);
  y += 64;
  textFit(`今日订单：收购商高价收【${G.orderTag}】类，${tuning.order_bonus} 倍`, 16, y + 12, W - 170, 21, C.neon);
  text(`口碑 ${G.rep}`, W - 16, y + 12, 21, C.ok, "right");
  y += 34;
  text("营业时间", 16, y + 10, 20, C.muted);
  const total = tuning.time_per_day + upValue("sign");
  const pw = (W - 140 - 16) / total;
  for (let i = 0; i < total; i++) rrect(120 + i * pw, y + 4, pw - 4, 12, 6, i < G.timeLeft ? C.neon : "#3a302a");
```

```
  if (G.intel > 0) text(`情报 ×${G.intel}`, W - 16, y + 10, 18, C.lcd, "right");
  return y + 30;
}
function drawScene(nav, x, y, w, h) {
  const d = G.deal;
  ctx.save();
  rrect(x, y, w, h, 18, "#2a1d15");
  ctx.clip();
  image("shop_bg", x, y, w, h, { cover: true });
  ctx.fillStyle = "rgba(0,0,0,0.25)"; ctx.fillRect(x, y, w, h);
  // 顾客：进门时从右边滑进来
  const k = ease.out(anim("arrive"));
  const sx = shakeX("figure", 12);
  const pw = Math.min(w * 0.42, h * 0.95);
  image(d.customer.art, x + 10 + (1 - k) * 120 + sx, y + h - pw * 1.05, pw, pw * 1.05, { alignY: 1 });
  ctx.restore();
  // 名字 + 耐心
  rrect(x + 12, y + 12, 220, 70, 12, "rgba(0,0,0,0.55)");
  textFit(d.customer.name, x + 24, y + 34, 200, 24, C.text, "left", true);
  const maxP = tuning.patience_start * 1.3;
  for (let i = 0; i < 5; i++) {
    const on = d.patience / maxP > i / 5;
    rrect(x + 24 + i * 38, y + 58, 32, 12, 6, on ? C.ok : "#3a302a");
  }
  // 对话气泡：顾客讲的来历
  const bx = x + w * 0.4, bw = w * 0.58;
  const bh = Math.min(h - 70, paraHeight(`“${d.story}”`, bw - 32, 24) + 30);
  ctx.globalAlpha = k;
  rrect(bx, y + 20, bw, bh, 20, C.paper);
  ctx.save(); ctx.beginPath(); ctx.rect(bx, y + 20, bw, bh); ctx.clip();
  para(`“${d.story}”`, bx + 16, y + 34, bw - 32, 24, C.ink);
  ctx.restore();
  ctx.globalAlpha = 1;
  if (G.giftQueue.length) button(`📦 包裹 ×${G.giftQueue.length}`, x + w - 200, y + h - 64, 188, 52, () => nav.openGift
(), { size: 22 });
}
function drawTicket(x, y, w, h) {
  const d = G.deal;
  const k = anim("arrive");
  ctx.globalAlpha = Math.min(1, k * 1.5);
  rrect(x, y, w, h, 18, C.paper, C.paperDark, 3);
  image(d.item.art, x + 16, y + 14, 96, 96);
  text("回 收 单", x + 128, y + 26, 18, C.inkSoft);
  textFit(d.item.name, x + 128, y + 60, w - 300, 32, C.ink, "left", true);
  const hot = d.item.tag === G.orderTag;
  const tag = G.tagNote ? `【${d.item.tag}】${G.tagNote}` : `【${d.item.tag}】${hot ? "今日订单！" : ""}  市场参考价 ¥${d.i
tem.base}`;
  textFit(tag, x + 128, y + 94, w - 300, 20, G.tagNote ? "#2e9b5f" : hot ? C.stamp : C.inkSoft);
  // 要价印章
  const ps = 1 + 0.15 * (1 - anim("ask")) * (anim("ask") < 1 ? 1 : 0);
  ctx.save();
  ctx.translate(x + w - 80, y + 60); ctx.scale(ps, ps); ctx.rotate(-0.05);
```

```
  rrect(-62, -44, 124, 88, 8, "rgba(200,55,45,0.08)", C.stamp, 3);
  text("要价", 0, -16, 24, C.stamp, "center", true);
  textFit(`¥${d.ask}`, 0, 18, 116, 28, C.stamp, "center", true);
  ctx.restore();
  // 线索
  let cy = y + 124;
  d.clues.forEach((c, i) => {
    if (c.revealed) {
      rrect(x + 14, cy, w - 28, 60, 12, "#f8efd9", "#c9b48d", 2);
      textFit(`✓ ${c.part}：${c.text}`, x + 30, cy + 30, w - 60, 22, C.ink);
    } else {
      rrect(x + 14, cy, w - 28, 60, 12, C.paperDark, "#a8916b", 2);
      text(`◎  检查${c.part}`, x + 30, cy + 30, 26, C.ink);
      text(G.intel > 0 ? "情报" : `耗时 ${tuning.time_per_clue}`, x + w - 30, cy + 30, 20, C.inkSoft, "right");
      if (G.dealOpen) hit(x + 14, cy, w - 28, 60, () => revealClue(i));
    }
    cy += 70;
  });
  const exposed = d.exposed;
  rrect(x + 14, cy, w - 28, 60, 12, exposed ? "rgba(76,209,138,0.15)" : "rgba(200,55,45,0.06)", exposed ? C.ok : C.stam
p, 2);
  text(exposed ? "已揭穿：顾客说了实话" : "质疑来历：“这东西真是你的？”", x + w / 2, cy + 30, 26,
    exposed ? "#2e9b5f" : C.stamp, "center");
  if (G.dealOpen && !exposed) hit(x + 14, cy, w - 28, 60, () => {
    const r = challenge();
    if (r && (r.mood === "caught" || r.mood === "confess")) kick("ask"); else kick("figure", 500);
  });
  ctx.globalAlpha = 1;
}
function drawRegister(x, y, w, h) {
  const d = G.deal;
  rrect(x, y, w, h, 22, C.metal, "#3a322c", 2);
  const open = G.dealOpen;
  // − 显示屏 +
  button("−", x + 14, y + 14, 92, 110, () => stepOffer(-1), { fill: "#2e2722", color: C.text, size: 48, radius: 46, disa
bled: !open });
  button("+", x + w - 106, y + 14, 92, 110, () => stepOffer(1), { fill: "#2e2722", color: C.text, size: 48, radius: 46,
disabled: !open });
  rrect(x + 118, y + 14, w - 236, 110, 12, C.lcdBg, "#24402f", 2);
  text("你的出价", x + 134, y + 34, 18, "#4f9a6d");
  for (let i = 0; i < 3; i++) {   // 剩余砍价轮数
    ctx.beginPath(); ctx.arc(x + w - 150 - i * 24, y + 34, 8, 0, Math.PI * 2);
    ctx.fillStyle = i < d.roundsLeft ? C.lcd : "#24402f"; ctx.fill();
  }
  const pulse = 1 + 0.06 * (1 - anim("offer")) * (anim("offer") < 1 ? 1 : 0);
  ctx.save(); ctx.translate(x + w / 2, y + 80); ctx.scale(pulse, pulse);
  text(`¥ ${G.offer}`, 0, 0, 50, G.offer > G.cash ? C.bad : C.lcd, "center", true);
  ctx.restore();
  // 3/5/7/9 成
  const ratios = tuning.quick_offer_ratios;
  const cw = (w - 28 - 10 * (ratios.length - 1)) / ratios.length;
  ratios.forEach((r, i) => button(`${Math.round(r * 10)} 成`, x + 14 + i * (cw + 10), y + 136, cw, 52,
    () => { quickOffer(r); kick("offer"); }, { fill: "#2e2722", color: C.lcd, stroke: "#24402f", size: 24, disabled: !op
en }));
```

```
  button("出 价", x + 14, y + 198, w - 28, 56, () => {
    const r = makeOffer();
    if (r === "counter") { kick("figure", 500); kick("ask"); }
  }, { size: 30, disabled: !open });
  const hw = (w - 38) / 2;
  button(`按要价收 ¥${d.ask}`, x + 14, y + 262, hw, 46, () => takeAsk(),
    { fill: "rgba(0,0,0,0)", stroke: C.ok, color: C.ok, size: 22, disabled: !open || d.ask > G.cash });
  button("不收了", x + 24 + hw, y + 262, hw, 46, () => refuse(),
    { fill: "rgba(0,0,0,0)", stroke: C.bad, color: C.bad, size: 22, disabled: !open });
}
// ===== src/state.js =====
// 游戏状态和全部规则：存档、开店流程、买卖、打烊结算、贷款、包裹、员工外出、升级、抽卡、成就。
// 界面（scenes/）只读这里的状态、调这里的函数，不自己改数值。
import tuning from "../build/data/tuning.json";
import catalog from "../build/data/catalog.json";
import customersData from "../build/data/customers.json";
import personalitiesData from "../build/data/personalities.json";
import gifts from "../build/data/gifts.json";
import staffData from "../build/data/staff.json";
import meta from "../build/data/meta.json";
import { Deal, pick, rand, randi } from "./deal.js";
import { storage, audio, now } from "./platform.js";
import { floatText, C } from "./ui.js";
export { tuning, catalog, gifts, staffData, meta };
const archetypes = customersData.archetypes;
const personas = personalitiesData.types;
const SAVE_KEY = "standby_salvage_save_v1";
/** 全局状态。G.deal 是当前顾客（不存档，读档时重新来一位）。 */
export const G = {
  day: 1, cash: tuning.start_cash, rep: 0, inventory: [], timeLeft: tuning.time_per_day, orderTag: "",
  runState: "playing", emergencyUsed: false, extraDue: {},
  collection: {}, giftQueue: [], giftsToday: 0, intel: 0, lastDaily: "",
  settings: { music: 0.6, sfx: 1 },
  staff: [], upgrades: {}, skins: {}, stats: {}, ach: {},
  // 以下不存档
  deal: null, dealOpen: false, offer: 0, log: { text: "", color: C.text }, result: null,
  achToasts: [], tagNote: null,
};
// ---------------- 存档 ----------------
const SAVED = ["day", "cash", "rep", "inventory", "timeLeft", "orderTag", "runState", "emergencyUsed", "extraDue",
  "collection", "giftQueue", "giftsToday", "intel", "lastDaily", "settings", "staff", "upgrades", "skins", "stats", "ac
h"];
export function save() {
  const d = {};
  for (const k of SAVED) d[k] = G[k];
  storage.set(SAVE_KEY, d);
}
/** 读档。返回 true = 有存档。 */
export function load() {
  const d = storage.get(SAVE_KEY);
  if (!d) return false;
```

```
  for (const k of SAVED) if (d[k] !== undefined) G[k] = d[k];
  audio.setVolume(G.settings.music, G.settings.sfx);
  return true;
}
// ---------------- 第一次打开 ----------------
export function newPlayer() {
  for (const s of meta.gacha.skins.slice(0, 4)) G.skins[s.look] = true;   // 送 4 个普通造型
  for (let i = 0; i < staffData.start_staff; i++) addStaff();
  G.giftQueue.push("small");   // 见面礼
}
// ---------------- 升级 ----------------
export const upLevel = (id) => G.upgrades[id] || 0;
/** 某个升级当前生效的数值（没买为 0）。 */
export function upValue(id) {
  const u = meta.upgrades.find((x) => x.id === id);
  const lv = upLevel(id);
  return lv > 0 ? u.levels[lv - 1].value : 0;
}
export function buyUpgrade(id) {
  const u = meta.upgrades.find((x) => x.id === id);
  const lv = upLevel(id);
  if (lv >= u.levels.length) return false;
  const cost = u.levels[lv].cost;
  if (G.cash < cost) { floatText("钱不够", C.bad); return false; }
  G.cash -= cost;
  G.upgrades[id] = lv + 1;
  bump("upgrades");
  setStat("maxed_upgrades", meta.upgrades.filter((x) => upLevel(x.id) >= x.levels.length).length);
  audio.sfx("coin");
  floatText(`${u.name} 升到 ${lv + 1} 级`, C.ok);
  save();
  return true;
}
const dayLength = () => tuning.time_per_day + upValue("sign");
// ---------------- 成就 ----------------
export function bump(stat, n = 1) { G.stats[stat] = (G.stats[stat] || 0) + n; checkAch(); }
export function setMax(stat, v) { if (v > (G.stats[stat] || 0)) { G.stats[stat] = v; checkAch(); } }
function setStat(stat, v) { G.stats[stat] = v; checkAch(); }
export function checkAch() {
  for (const a of meta.achievements) {
    if (G.ach[a.id]) continue;
    if ((G.stats[a.stat] || 0) >= a.goal) {
      G.ach[a.id] = Date.now();
      G.achToasts.push({ a, t0: now() });
      audio.sfx("win");
    }
  }
}
function refreshCollectionStats() {
  setStat("collection", Object.keys(G.collection).length);
```

```
  const bySeries = {};
  for (const c of gifts.collectibles) {
    const s = bySeries[c.series] || (bySeries[c.series] = { have: 0, all: 0 });
    s.all++; if (G.collection[c.id]) s.have++;
  }
  setStat("series_done", Object.values(bySeries).filter((s) => s.have === s.all).length);
  setStat("legend_items", gifts.collectibles.filter((c) => c.rarity >= 5 && G.collection[c.id]).length);
  setStat("skins", Object.keys(G.skins).length);
}
// ---------------- 开店流程 ----------------
export function setLog(text, color = C.text) { G.log = { text, color }; }
export function startDay() {
  audio.bgm("shop");
  G.runState = "playing";
  G.timeLeft = dayLength();
  G.orderTag = pick(tuning.order_tags);
  setMax("best_day_reached", G.day);
  const h = new Date().getHours();
  if (h < 5) setMax("night", 1);
  nextCustomer();
  floatText(`第 ${G.day} 天开张`);
}
export function nextCustomer() {
  if (G.timeLeft < tuning.time_per_customer + 1) { endDay(); return; }
  G.timeLeft -= tuning.time_per_customer;
  const d = new Deal(tuning, catalog, archetypes, personas, G.rep);
  d.patience *= 1 + upValue("charm");
  G.deal = d;
  G.dealOpen = true;
  G.offer = Math.max(10, Math.min(d.ask, Math.max(G.cash, 10)));  // 出价屏先显示顾客要价
  G.tagNote = null;
  setLog(`“${d.line("greet")}”`);
  if (upLevel("appraiser")) d.reveal(randi(0, d.clues.length - 1));   // 老鉴定师免费看一处
  audio.sfx("door");
}
export function revealClue(i) {
  const d = G.deal;
  if (!G.dealOpen || d.clues[i].revealed) return;
  const free = G.intel > 0 || Math.random() < upValue("lens");
  if (!free && G.timeLeft < tuning.time_per_clue) { setLog("今天没时间细看了。", C.bad); return; }
  if (G.intel > 0) { G.intel--; floatText("用掉一条情报，不耗时"); }
  else if (free) floatText("放大镜：一眼看穿，不耗时");
  else G.timeLeft -= tuning.time_per_clue;
  d.reveal(i);
  audio.sfx("clue");
}
export function challenge() {
  const d = G.deal;
  if (!G.dealOpen || d.exposed) return null;
  const r = d.challenge();
```

```
  setLog(r.text, r.mood === "caught" || r.mood === "confess" ? C.ok : C.bad);
  if (r.mood === "caught" || r.mood === "confess") {
    G.tagNote = `揭穿了：它其实只值 ¥${d.ask} 左右`;
    changeRep(1, "揭穿了骗子");
    bump("liars");
    audio.sfx("caught");
  } else {
    if (r.mood === "honest") bump("wrong_accuse");
    audio.sfx("angry");
  }
  if (d.patience <= 0) { closeDeal("顾客扭头就走了。"); return r; }
  G.offer = Math.min(G.offer, d.ask);
  return r;
}
export function quickOffer(ratio) {
  if (!G.dealOpen) return;
  const d = G.deal;
  G.offer = clampOffer(Math.round(d.ask * ratio / 10) * 10);
  G.lastRatio = ratio;
}
export function stepOffer(dir) {
  if (!G.dealOpen) return;
  const d = G.deal;
  const step = Math.max(10, Math.round(d.ask * tuning.offer_step_ratio / 10) * 10);
  G.offer = clampOffer(G.offer + dir * step);
  G.lastRatio = null;
}
const clampOffer = (v) => Math.max(10, Math.min(v, Math.min(Math.max(G.cash, 10), G.deal.ask)));
/** 出价，返回结果 accept / counter / leave（界面用来做动画） */
export function makeOffer() {
  if (!G.dealOpen) return null;
  if (G.offer > G.cash) { setLog("钱不够了。", C.bad); return null; }
  const d = G.deal;
  const r = d.offer(G.offer);
  if (r.result === "accept") {
    setLog(r.text, C.ok);
    if (G.lastRatio && G.lastRatio <= 0.31) bump("lowball");
    buy(G.offer);
  } else if (r.result === "leave") {
    setLog(r.text, C.bad);
    closeDeal(r.text);
  } else {
    setLog(r.text);
    audio.sfx("counter");
    G.offer = Math.min(G.offer, d.ask);
  }
  return r.result;
}
export function takeAsk() {
  const d = G.deal;
```

```
  if (G.dealOpen && d.ask <= G.cash) { setLog(`“${d.line("accept")}”`, C.ok); buy(d.ask); }
}
export function refuse() {
  if (G.dealOpen) closeDeal(`你摆摆手，${G.deal.customer.name} 带着${G.deal.item.name}走了。`);
}
function changeRep(delta, why) {
  G.rep += delta;
  setMax("best_rep", G.rep);
  floatText(`街坊口碑 ${delta > 0 ? "+" : ""}${delta} · ${why}`, delta > 0 ? C.ok : C.bad);
}
function buy(price) {
  const d = G.deal;
  G.cash -= price;
  audio.sfx("coin");
  G.inventory.push({ name: d.item.name, paid: price, value: d.trueValue, leaky: !!d.item.leaky, tag: d.item.tag });
  if (!d.lying) {
    const ratio = price / Math.max(1, d.trueValue);
    if (ratio >= tuning.rep_fair_ratio) changeRep(1, "给了公道价");
    else if (ratio < tuning.rep_lowball_ratio) changeRep(-1, "压价太狠");
  }
  const p = d.profit(price);
  bump("deals");
  setMax("best_profit", p);
  setMax("worst_loss", -p);
  if (d.item.tag === G.orderTag) bump("order_items");
  const lines = [`${d.item.name}  付出 ¥${price}`, `收购商估价 ¥${d.trueValue}`];
  if (d.item.tag === G.orderTag) lines.push(`符合今日订单，打烊时按 ${tuning.order_bonus} 倍收`);
  if (d.lying) {
    const lie = d.clues.find((c) => c.isLie);
    lines.push(`顾客说了谎：${lie ? lie.part + lie.text : ""}`);
  }
  if (d.item.leaky) lines.push("注意：这是会漏液的问题货，可能弄坏仓库里别的货");
  showResult("成交", p >= 0 ? `+¥${p}` : `-¥${-p}`, p >= 0 ? C.ok : C.bad, lines.join("\n"));
}
function closeDeal(textLine) {
  audio.sfx("leave");
  showResult("没谈成", "¥0", C.muted, `${textLine}\n\n它其实值 ¥${G.deal.trueValue}`);
}
function showResult(title, big, bigColor, body, btn, btn2 = null) {
  G.dealOpen = false;
  G.result = { title, big, bigColor, body, t0: now(), btn2,
    btn: btn || (G.timeLeft >= tuning.time_per_customer + 1 ? "下一位顾客" : "打烊结算") };
  setMax("best_cash", G.cash);
  save();
}
/** 结果卡片上的主按钮 */
export function resultNext() {
  G.result = null;
  if (G.runState === "bankrupt") { takeEmergencyLoan(); return; }
  if (G.runState === "playing" && isBroke()) { showBankrupt(); return; }
```

```
  if (G.runState === "day_end") startDay();
  else if (G.runState === "lost" || G.runState === "won") restart();
  else nextCustomer();
}
/** 结果卡片上的第二个按钮（破产时“不借了”） */
export function resultAlt() {
  G.result = null;
  G.runState = "lost";
  restart();
}
const isBroke = () => G.cash < 50 && G.inventory.length === 0;
function nextDue(fromDay) {
  for (let d = fromDay; d <= tuning.run_days; d++) {
    const amount = (tuning.loan_due[String(d)] || 0) + (G.extraDue[String(d)] || 0);
    if (amount > 0) return { day: d, amount };
  }
  return null;
}
export function upcomingDue() { return nextDue(G.day); }
function showBankrupt() {
  bump("bankrupts");
  if (!G.emergencyUsed) {
    const due = nextDue(G.day);
    G.runState = "bankrupt";
    showResult("破产了", `¥${G.cash}`, C.bad,
      `钱花光了，仓库也空了。\n\n银行愿意借你一笔应急款 ¥${tuning.emergency_loan}，\n第 ${due ? due.day : tuning.run_days} 天还款时要多还 ¥${tuning.emergency_repay}。\n（每局只能借一次）`,
      "借应急款", "不借了，重新开始");
  } else {
    G.runState = "lost";
    showResult("破产了", `¥${G.cash}`, C.bad, "钱花光了，应急款也借过了……\n店只能先关门了。\n\n收藏品、升级和员工都会保留，下一局再来！", "从头再来");
  }
  audio.sfx("lose");
}
function takeEmergencyLoan() {
  G.emergencyUsed = true;
  G.cash += tuning.emergency_loan;
  const due = nextDue(G.day);
  const key = String(due ? due.day : tuning.run_days);
  G.extraDue[key] = (G.extraDue[key] || 0) + tuning.emergency_repay;
  G.runState = "playing";
  floatText(`借到应急款 ¥${tuning.emergency_loan}`);
  audio.sfx("coin");
  nextCustomer();
}
function restart() {
  G.emergencyUsed = false;
  G.extraDue = {};
  G.day = 1;
  G.cash = tuning.start_cash;
  G.rep = 0;
```

```
  G.inventory = [];
  startDay();
}
function endDay() {
  const lines = [];
  const leaky = G.inventory.filter((it) => it.leaky);
  if (leaky.length && G.inventory.length > 1 && !upLevel("safe")) {
    const victims = G.inventory.filter((it) => !it.leaky);
    if (victims.length) {
      const v = pick(victims);
      const loss = Math.round(v.value * tuning.leak_damage);
      v.value -= loss;
      lines.push(`漏液弄坏了${v.name}，少卖 ¥${loss}`);
    }
  }
  let total = 0, paid = 0, bonus = 0;
  const buyer = 1 + upValue("buyer");
  for (const it of G.inventory) {
    let value = Math.round(it.value * buyer);
    if (it.tag === G.orderTag) { const extra = Math.round(value * (tuning.order_bonus - 1)); bonus += extra; value += ex
tra; }
    total += value; paid += it.paid;
  }
  G.cash += total;
  const net = total - paid;
  setMax("best_day", net);
  lines.unshift(`收了 ${G.inventory.length} 件货，花了 ¥${paid}`);
  lines.splice(1, 0, `收购商一共付了 ¥${total}`);
  if (bonus > 0) lines.splice(2, 0, `其中订单加价 ¥${bonus}`);
  G.inventory = [];
  G.runState = "day_end";
  const due = (tuning.loan_due[String(G.day)] || 0) + (G.extraDue[String(G.day)] || 0);
  if (due > 0) {
    if (G.cash >= due) { G.cash -= due; lines.push(`\n还贷 ¥${due} ✓ 收债人点点头走了`); bump("loans_paid"); }
    else { lines.push(`\n今天要还贷 ¥${due}，你只有 ¥${G.cash}……\n收债人把店门锁上了。`); G.runState = "lost"; }
  }
  lines.push(`现金 ¥${G.cash}  口碑 ${G.rep}`);
  if (G.runState === "day_end" && G.day >= tuning.run_days) {
    G.runState = "won";
    bump("wins");
    if (G.emergencyUsed) bump("comebacks");
    lines.push("\n贷款全部还清，这家店归你了。\n升级、员工、收藏品都保留，下一局会更轻松。");
  } else if (G.runState === "day_end") {
    const nd = nextDue(G.day + 1);
    if (nd) lines.push(`下次还贷：第 ${nd.day} 天 ¥${nd.amount}`);
  }
  let title = `第 ${G.day} 天 打烊`, btn = `开始第 ${G.day + 1} 天`;
  if (G.runState === "lost") { title = "店被收走了"; btn = "从头再来"; }
  if (G.runState === "won") { title = "七天撑过来了！"; btn = "再开一局"; }
  if (G.runState === "day_end") G.day++;
  G.giftsToday = 0;
```

```
  audio.sfx(G.runState === "lost" ? "lose" : G.runState === "won" ? "win" : "day_end");
  audio.bgm(G.runState === "day_end" ? "closing" : "");
  showResult(title, net >= 0 ? `+¥${net}` : `-¥${-net}`, net >= 0 ? C.ok : C.bad, lines.join("\n"), btn);
}
// ---------------- 包裹 ----------------
export function addGift(tier) {
  if (G.giftsToday >= gifts.daily_cap) { floatText("今天的包裹够多了，员工把它放进了仓库", C.muted); return; }
  G.giftsToday++;
  if (tier === "small" && Math.random() < gifts.weird_chance) tier = "weird";
  G.giftQueue.push(tier);
  audio.sfx("notify");
  save();
}
function pickOutcome(tier) {
  const w = (r) => r.weight * (tier === "big" && (r.min || 0) < 0 ? 0.5 : 1);
  const rows = gifts.outcomes;
  let roll = Math.random() * rows.reduce((s, r) => s + w(r), 0);
  for (const r of rows) { roll -= w(r); if (roll <= 0) return r; }
  return rows[0];
}
function rollCollectible(tier) {
  const pool = gifts.collectibles;
  const ws = pool.map((c) => 1 / c.rarity ** (tier === "big" ? 1.2 : 2));
  let roll = Math.random() * ws.reduce((a, b) => a + b, 0);
  for (let i = 0; i < pool.length; i++) { roll -= ws[i]; if (roll <= 0) return pool[i]; }
  return pool[0];
}
const pickText = (o) => pick(o.text || [""]);
/** 拆开队首的包裹。返回 {title, body, art, good}。 */
export function openGift() {
  const tier = G.giftQueue.shift();
  if (!tier) return null;
  const bonus = gifts.tiers[tier].bonus;
  const o = pickOutcome(tier);
  let good = true, body = "", art = gifts.tiers[tier].art;
  if (o.kind === "cash") {
    const amount = Math.round(randi(o.min, o.max) * (o.min > 0 ? bonus : 1));
    G.cash += amount; good = amount >= 0;
    body = `${pickText(o)}\n\n${amount >= 0 ? "+" : "-"} ¥${Math.abs(amount)}`;
  } else if (o.kind === "collectible") {
    const c = rollCollectible(tier);
    const fresh = !G.collection[c.id];
    G.collection[c.id] = (G.collection[c.id] || 0) + 1;
    art = c.art || "col_" + c.id;
    body = `${fresh ? "新" : "重复的"}收藏品：${c.name} ${"★".repeat(c.rarity)}\n\n${c.desc}`;
    if (!fresh) { const refund = 5 * c.rarity; G.cash += refund; body += `\n\n（重复的卖给了收藏贩子 +¥${refund}）`; }
    refreshCollectionStats();
  } else if (o.kind === "intel") {
    G.intel += o.count;
    body = `${pickText(o)}\n\n情报 +${o.count}：看线索免费、不耗时`;
```

```
  } else if (o.kind === "rep") {
    G.rep += o.amount;
    setMax("best_rep", G.rep);
    body = `${pickText(o)}\n\n街坊口碑 +${o.amount}`;
  } else { good = false; body = pickText(o); }
  bump("gifts");
  setMax("best_cash", G.cash);
  audio.sfx(good ? "gift" : "bad_gift");
  save();
  return { title: good ? "拆开了！" : "呃……", body, art, good };
}
// ---------------- 员工 ----------------
let staffSeq = 0;
function addStaff() {
  const used = new Set(G.staff.map((s) => s.name));
  const name = pick(staffData.names.filter((n) => !used.has(n))) || `员工${G.staff.length + 1}`;
  const owned = Object.keys(G.skins);
  const usedLooks = new Set(G.staff.map((s) => s.look));
  const look = pick(owned.filter((l) => !usedLooks.has(l))) || pick(owned) || "adv_01";   // 先用没人穿的造型
  G.staff.push({ id: `s${Date.now().toString(36)}${staffSeq++}`, name, look, status: "idle" });
  setStat("staff", G.staff.length);
}
export const nextHireCost = () => G.staff.length < staffData.max_staff ? staffData.hire_cost[G.staff.length] : null;
export function hire() {
  const cost = nextHireCost();
  if (cost === null) return false;
  if (G.cash < cost) { floatText("钱不够", C.bad); return false; }
  G.cash -= cost;
  addStaff();
  audio.sfx("door");
  floatText(`新员工 ${G.staff[G.staff.length - 1].name} 报到！`, C.ok);
  save();
  return true;
}
export const placesOpen = () => staffData.places.filter((p) => p.unlock_day <= Math.max(G.day, G.stats.best_day_reached
|| 1));
export function tripMinutes(place) { return place.minutes * (1 - upValue("bed")); }
export function sendStaff(staff, place) {
  staff.status = "away";
  staff.place = place.id;
  staff.departAt = now();
  staff.returnAt = now() + tripMinutes(place) * 60000;
  bump("trips");
  if (place.id === "mansion") bump("mansion_trips");
  audio.sfx("door");
  save();
}
/** 每帧调用：到点的员工变成“带着汇报回来了”。返回这次新回来的人。 */
export function tickStaff() {
  const back = [];
  for (const s of G.staff) {
```

```
    if (s.status === "away" && now() >= s.returnAt) {
      const place = staffData.places.find((p) => p.id === s.place) || staffData.places[0];
      s.status = "back";
      s.report = makeReport(place);
      let tier = place.tier;
      if (tier === "small" && Math.random() < upValue("map") + staffLevel(s).big_chance) tier = "big";
      s.gift = tier;
      s.dilemma = Math.random() < staffData.dilemma_chance ? makeDilemma() : null;
      back.push(s);
    }
  }
  if (back.length) save();
  return back;
}
function makeReport(place) {
  const scenes = [...place.scenes].sort(() => Math.random() - 0.5).slice(0, 2);
  const lines = [pick(staffData.openings), `这次去了${place.name}。`, `${scenes[0]}。`];
  if (Math.random() < 0.6) lines.push(pick(staffData.events));
  lines.push(`${scenes[1]}。`, `最后${pick(place.finds)}。`, pick(staffData.endings));
  return lines.join("\n");
}
function makeDilemma() {
  const d = pick(staffData.dilemmas);
  const cost = Math.round(d.cost * rand(0.8, 1.3) / 10) * 10;
  return { ...d, cost, text: d.text.replace("{cost}", cost) };
}
/** 回答员工带回的小选择。返回结果文字。 */
export function answerDilemma(staff, yes) {
  const d = staff.dilemma;
  staff.dilemma = null;
  if (!yes) return d.no.text;
  if (G.cash < d.cost) return "钱不够……那就算了吧。";
  G.cash -= d.cost;
  bump("dilemma_yes");
  if (d.yes.rep) { G.rep += d.yes.rep; setMax("best_rep", G.rep); }
  const [lo, hi] = d.yes.cash;
  const got = hi > 0 ? Math.round(rand(lo, hi) / 10) * 10 : 0;
  G.cash += got;
  const r = staffData.dilemma_results;
  const result = got > d.cost * 1.2 ? r.good : got >= d.cost * 0.8 ? r.even : r.bad;
  save();
  return d.yes.text.replace("{result}", hi > 0 ? `${result}（花 ¥${d.cost}，卖了 ¥${got}）` : "");
}
/** 员工等级：按完成的探险次数（staff.json 的 levels）。 */
export function staffLevel(s) {
  let lv = staffData.levels[0];
  for (const l of staffData.levels) if ((s.trips || 0) >= l.trips) lv = l;
  return lv;
}
/** 听完汇报：员工回到店里歇着，探险次数 +1（可能升级），包裹进队列。 */
```

```
export function finishReport(staff) {
  const before = staffLevel(staff);
  staff.trips = (staff.trips || 0) + 1;
  const after = staffLevel(staff);
  if (after !== before) { floatText(`${staff.name} 升级成了「${after.title}」！`, C.ok); audio.sfx("win"); }
  setMax("staff_trips_max", staff.trips);
  staff.status = "idle";
  staff.report = null;
  addGift(staff.gift || "small");
  staff.gift = null;
  save();
}
// ---------------- 每日包裹 ----------------
/** 每天第一次打开：店里一位员工带来一个包裹。返回 {staff, text} 或 null。 */
export function dailyGift(today = new Date().toDateString()) {
  if (G.lastDaily === today) return null;
  G.lastDaily = today;
  const s = pick(G.staff.filter((x) => x.status !== "away")) || G.staff[0];
  G.giftQueue.push(staffData.daily_gift.tier);
  save();
  return { staff: s, text: pick(staffData.daily_gift.text) };
}
// ---------------- 抽皮肤 ----------------
function rollSkin() {
  const rs = meta.gacha.rarities;
  let roll = Math.random() * rs.reduce((s, r) => s + r.weight, 0);
  let rar = rs[0];
  for (const r of rs) { roll -= r.weight; if (roll <= 0) { rar = r; break; } }
  const pool = meta.gacha.skins.filter((s) => s.rarity === rar.id);
  return pick(pool.length ? pool : meta.gacha.skins);
}
/** 抽 n 次，返回结果列表 [{skin, fresh, refund}]；钱不够返回 null。 */
export function draw(n) {
  const cost = n >= 10 ? meta.gacha.ten_cost : meta.gacha.cost * n;
  if (G.cash < cost) { floatText("钱不够", C.bad); return null; }
  G.cash -= cost;
  const out = [];
  for (let i = 0; i < n; i++) {
    const skin = rollSkin();
    const fresh = !G.skins[skin.look];
    G.skins[skin.look] = true;
    if (!fresh) G.cash += meta.gacha.dupe_refund;
    out.push({ skin, fresh, refund: fresh ? 0 : meta.gacha.dupe_refund });
    bump("draws");
    if (skin.rarity === "ssr") bump("ssr");
  }
  refreshCollectionStats();
  audio.sfx("gift");
  save();
  return out;
```

```
}
export function setLook(staff, look) { if (G.skins[look]) { staff.look = look; save(); } }
// ===== src/ui.js =====
// 画面工具：每一帧整屏重画（即时模式），按钮在画的时候登记点击区域。
import { ctx, W, H, img, audio, now } from "./platform.js";
export const C = {
  bgTop: "#2b1e17", bgBottom: "#0e0b09", neon: "#ffb347", pink: "#ff6b8b", text: "#f6ede2", muted: "#a8998a",
  paper: "#efe1c6", paperDark: "#d9c6a3", ink: "#3a2618", inkSoft: "#7a5d44", stamp: "#c8372d", ok: "#4cd18a",
  bad: "#ff5d5d", lcdBg: "#0d1a13", lcd: "#7dffb2", metal: "#1c1815",
};
const FONT = '"PingFang SC","Hiragino Sans GB","Microsoft YaHei","Noto Sans SC",sans-serif';
// ---------------- 点击区域 ----------------
let hits = [];          // 这一帧登记的按钮 [{x,y,w,h,fn,layer}]
let layer = 0;          // 弹窗会提高层级，下面的按钮就点不到
export function beginFrame() { hits = []; layer = 0; }
export function pushLayer() { layer++; }
/** 登记一块可点击区域。弹窗打开时，只有最高层的区域有效。 */
export function hit(x, y, w, h, fn) { hits.push({ x, y, w, h, fn, layer }); }
export function tap(x, y) {
  const top = hits.reduce((m, r) => Math.max(m, r.layer), 0);
  for (let i = hits.length - 1; i >= 0; i--) {
    const r = hits[i];
    if (r.layer === top && x >= r.x && x <= r.x + r.w && y >= r.y && y <= r.y + r.h) {
      audio.sfx("tap");
      r.fn();
      return true;
    }
  }
  return false;
}
// ---------------- 基础图形 ----------------
export function rrect(x, y, w, h, r, fill, stroke, lw = 2) {
  ctx.beginPath();
  r = Math.min(r, w / 2, h / 2);
  ctx.moveTo(x + r, y);
  ctx.arcTo(x + w, y, x + w, y + h, r);
  ctx.arcTo(x + w, y + h, x, y + h, r);
  ctx.arcTo(x, y + h, x, y, r);
  ctx.arcTo(x, y, x + w, y, r);
  ctx.closePath();
  if (fill) { ctx.fillStyle = fill; ctx.fill(); }
  if (stroke) { ctx.strokeStyle = stroke; ctx.lineWidth = lw; ctx.stroke(); }
}
export function vgrad(x, y, w, h, top, bottom) {
  const g = ctx.createLinearGradient(0, y, 0, y + h);
  g.addColorStop(0, top); g.addColorStop(1, bottom);
  ctx.fillStyle = g; ctx.fillRect(x, y, w, h);
}
export function font(size, bold = false) { ctx.font = `${bold ? "bold " : ""}${size}px ${FONT}`; }
/** 画一行文字。align: left / center / right。返回宽度。 */
```

```
export function text(s, x, y, size, color, align = "left", bold = false) {
  font(size, bold);
  ctx.fillStyle = color;
  ctx.textAlign = align;
  ctx.textBaseline = "middle";
  ctx.fillText(String(s), x, y);
  return ctx.measureText(String(s)).width;
}
/** 太长就截断加省略号 */
export function textFit(s, x, y, maxW, size, color, align = "left", bold = false) {
  font(size, bold);
  s = String(s);
  if (ctx.measureText(s).width > maxW) {
    while (s.length > 1 && ctx.measureText(s + "…").width > maxW) s = s.slice(0, -1);
    s += "…";
  }
  return text(s, x, y, size, color, align, bold);
}
const NO_LINE_START = "，。、；：？！）」』”…·";
const wrapCache = new Map();
/** 按宽度把文字拆成多行（中文逐字断，保留换行符）。 */
export function wrapLines(s, maxW, size, bold = false) {
  const key = `${size}|${bold}|${maxW}|${s}`;
  const cached = wrapCache.get(key);
  if (cached) return cached;
  font(size, bold);
  const out = [];
  for (const para of String(s).split("\n")) {
    let line = "";
    for (const ch of para) {
      // 标点不放在行首：宁可这一行稍微超出一点
      if (ctx.measureText(line + ch).width > maxW && line && !NO_LINE_START.includes(ch)) { out.push(line); line = ch; }
      else line += ch;
    }
    out.push(line);
  }
  if (wrapCache.size > 500) wrapCache.clear();
  wrapCache.set(key, out);
  return out;
}
/** 多行文字，返回占用的高度。 */
export function para(s, x, y, maxW, size, color, align = "left", lineH = 1.45, bold = false) {
  const lines = wrapLines(s, maxW, size, bold);
  const lh = size * lineH;
  const ax = align === "center" ? x + maxW / 2 : align === "right" ? x + maxW : x;
  lines.forEach((l, i) => text(l, ax, y + lh * i + lh / 2, size, color, align, bold));
  return lines.length * lh;
}
export function paraHeight(s, maxW, size, lineH = 1.45, bold = false) {
  return wrapLines(s, maxW, size, bold).length * size * lineH;
```

```
}
/** 按钮：圆角底 + 居中文字，登记点击。disabled 时变暗且点不了。 */
export function button(label, x, y, w, h, fn, opt = {}) {
  const { fill = C.neon, color = "#1a1208", stroke = null, size = 28, disabled = false, radius = 16, bold = true } = op
t;
  ctx.globalAlpha = disabled ? 0.4 : 1;
  rrect(x, y, w, h, radius, fill, stroke, 2);
  textFit(label, x + w / 2, y + h / 2 + 1, w - 16, size, color, "center", bold);
  ctx.globalAlpha = 1;
  if (!disabled) hit(x, y, w, h, fn);
}
/** 图片按比例放进框里（contain）。没加载好时画一个占位色块。 */
export function image(name, x, y, w, h, opt = {}) {
  const im = img(name);
  if (!im) {
    if (opt.placeholder) rrect(x, y, w, h, 12, opt.placeholder);
    return false;
  }
  const s = opt.cover ? Math.max(w / im.width, h / im.height) : Math.min(w / im.width, h / im.height);
  const dw = im.width * s, dh = im.height * s;
  const ax = opt.alignX ?? 0.5, ay = opt.alignY ?? 0.5;
  ctx.drawImage(im, x + (w - dw) * ax, y + (h - dh) * ay, dw, dh);
  return true;
}
/** 整屏暗色遮罩，并提高层级（下面的按钮点不到）。 */
export function dim(alpha = 0.72) {
  ctx.fillStyle = `rgba(3,2,2,${alpha})`;
  ctx.fillRect(0, 0, W, H);
  pushLayer();
  hit(0, 0, W, H, () => {});   // 吞掉点在空白处的点击
}
// ---------------- 浮动提示 ----------------
const floats = [];
export function floatText(s, color = C.neon) { floats.push({ s, color, t0: now() }); }
export function drawFloats() {
  const t = now();
  for (let i = floats.length - 1; i >= 0; i--) {
    const f = floats[i];
    const k = (t - f.t0) / 1600;
    if (k >= 1) { floats.splice(i, 1); continue; }
    ctx.globalAlpha = k < 0.5 ? 1 : 1 - (k - 0.5) * 2;
    const y = H * 0.32 - k * 70 - (floats.length - 1 - i) * 44;
    font(28, true);
    const w = ctx.measureText(f.s).width + 40;
    rrect(W / 2 - w / 2, y - 26, w, 52, 26, "rgba(0,0,0,0.65)");
    text(f.s, W / 2, y, 28, f.color, "center", true);
    ctx.globalAlpha = 1;
  }
}
// ---------------- 小动画 ----------------
const pulses = new Map();
```

```
/** 触发一个 0→1 的短动画，用 anim(key) 读进度（没在播放时返回 1）。 */
export function kick(key, ms = 350) { pulses.set(key, { t0: now(), ms }); }
export function anim(key) {
  const p = pulses.get(key);
  if (!p) return 1;
  const k = (now() - p.t0) / p.ms;
  if (k >= 1) { pulses.delete(key); return 1; }
  return k;
}
export const ease = { out: (k) => 1 - (1 - k) ** 3, back: (k) => 1 + 2.7 * (k - 1) ** 3 + 1.7 * (k - 1) ** 2 };
export function shakeX(key, amp = 10) { const k = anim(key); return k >= 1 ? 0 : Math.sin(k * 30) * amp * (1 - k); }
// ---------------- 拖动滚动 ----------------
/** 一块可滚动区域的状态；在 scene 里用 scroller.begin(...)/end() 包住内容。 */
export class Scroller {
  constructor() { this.y = 0; this.max = 0; this.drag = null; this.box = null; this.moved = false; }
  begin(x, y, w, h) {
    this.box = { x, y, w, h };
    ctx.save();
    ctx.beginPath(); ctx.rect(x, y, w, h); ctx.clip();
    ctx.translate(0, -this.y);
    this._hitStart = hits.length;
    return y;
  }
  /** contentH：内容总高度 */
  end(contentH) {
    ctx.restore();
    this.max = Math.max(0, contentH - this.box.h);
    this.y = Math.max(0, Math.min(this.max, this.y));
    // 把这块区域里登记的按钮坐标换算回屏幕坐标，并裁掉看不见的
    const { y: by, h: bh } = this.box;
    for (let i = this._hitStart; i < hits.length; i++) {
      const r = hits[i];
      r.y -= this.y;
      const top = Math.max(r.y, by), bot = Math.min(r.y + r.h, by + bh);
      if (bot <= top) { r.h = 0; r.w = 0; } else { r.h = bot - top; r.y = top; }
    }
    if (this.max > 0) {   // 滚动条
      const bx = this.box.x + this.box.w - 6;
      const th = Math.max(40, bh * bh / (contentH));
      rrect(bx, by + (bh - th) * (this.y / this.max), 4, th, 2, "rgba(255,255,255,0.25)");
    }
  }
  down(x, y) {
    const b = this.box;
    if (b && x >= b.x && x <= b.x + b.w && y >= b.y && y <= b.y + b.h) { this.drag = { y, start: this.y }; this.moved =
false; }
  }
  move(x, y) {
    if (!this.drag) return;
    if (Math.abs(y - this.drag.y) > 8) this.moved = true;
    this.y = Math.max(0, Math.min(this.max, this.drag.start - (y - this.drag.y)));
```

```
  }
  up() { const m = this.moved; this.drag = null; this.moved = false; return m; }
}
```